Republic of Tunisia

Ministry of Higher Education and Scientific Research

[University / Institute Name]

logo_university.png

logo_company.png

Final-Year Graduation Project Thesis

submitted in partial fulfilment of the requirements for the
National Engineering Diploma in Computer Science
Speciality: Data Engineering & Big Data Analytics

Device Behavior Scoring & Risk Classification

A CRISP-DM Pipeline on Microsoft Azure for Telematics Fleets

Author: Amine [Last Name]

Academic Supervisor: [Academic Supervisor]

Industry Supervisor: [Industry Supervisor]

Host Organization: Accent Tunisie

Academic Year: 2025–2026

To my parents, whose constant support and quiet sacrifices have shaped every step of this journey,

To my brothers and sisters, for the encouragement and the laughter that lightened the heavy weeks,

To my teachers, who turned curiosity into engineering discipline,

To the colleagues at Accent Tunisie, who welcomed me, trusted me with real data and treated my work as their own,

To every friend who listened patiently to my doubts and celebrated each small breakthrough,

I dedicate this work.

Acknowledgments

The work documented in this thesis is the result of six intense months during which many people invested time, expertise and goodwill on my behalf. It is a privilege to thank them here.

I am profoundly grateful to my academic supervisor, [Academic Supervisor], for the rigour of his guidance, the precision of his remarks and the patience with which he examined every iteration of the deliverables. His insistence on substance over form has meaningfully shaped the engineering posture of this project.

I extend my warmest thanks to my industry supervisor, [Industry Supervisor], who welcomed me at Accent Tunisie, granted me access to the operational telematics platform and trusted me with a refactoring scope of significant impact. The technical conversations we had on the warehouse design and on the production pipeline have considerably enriched the present work.

My sincere thanks go to the entire engineering team of Accent Tunisie who answered my questions on the data, the operational regimes of the fleet and the constraints of the Microsoft Azure environment on which the platform is deployed. The remote collaboration through SSH tunnelling and the controlled access to the cloud database would not have been as smooth without their day-to-day availability.

I would like to thank the members of the jury who have honoured this defence with their attention and whose remarks I will study carefully.

Finally, my deepest gratitude goes to my family and to my close friends, whose constant support has been the silent infrastructure of this entire project.

Abstract

This thesis describes the design, the implementation and the deployment of an end-to-end data engineering and machine learning platform built for the multi-tenant telematics fleet of Accent Tunisie. The host organization operates three MySQL databases that fulfil complementary roles in the lifecycle of a telemetry signal: a raw-stream database in which the frames pushed by GPS/IoT devices through SIM-card connectivity are stored as hexadecimal payloads; a decoded archive whose tables follow the convention `arch_xxxxx` and retain approximately the most recent six months of readable telemetry; and a master-data database whose tables follow the convention `user_xxxxx` and carry the per-tenant customers, vehicles and drivers. Although the company already exposes mature dashboards and standard reports on top of these sources, the analytical value of the historical archive remains largely untapped: there is no behavioural segmentation, no anomaly score and no continuous risk indicator that would let the operations team anticipate operational risk rather than only observe past activity. Following the CRISP-DM methodology, the project introduces an analytical layer that explicitly addresses this gap. Selected slices of the operational MySQL data are exported as `.sql` dump files, converted by a dedicated MySQL-to-PostgreSQL bridge into a PostgreSQL-importable representation, and loaded into a three-layer warehouse (staging, warehouse, marts) hosted on Microsoft Azure. The choice of PostgreSQL 16 as the analytical DBMS is justified by its strong support for analytical workloads, its ACID compliance, its advanced indexing, its scalability to large datasets, its extensibility and its first-class compatibility with the Python data ecosystem and standard BI tools. Because the project is conducted by a binomial of student engineers working from separate workstations, the analytical environment is hosted on an Azure virtual machine accessed through SSH tunnelling, with an IP allowlist enforced at the Network Security Group level. The thesis covers business understanding, data understanding through PostgreSQL profiling, data preparation through an explicit cleaning rule engine, feature engineering of 35 behavioural indicators at the device-month grain, the training of a per-tenant K-Means segmentation and an Isolation Forest risk score, and finally the production deployment through a Prefect flow scheduled by cron, served through a FastAPI service and a Streamlit dashboard.

Keywords: CRISP-DM, telematics, fleet analytics, advanced analytics, MySQL, PostgreSQL, data engineering, ETL, feature engineering, K-Means, Isolation Forest, Microsoft Azure, SSH tunnelling, monitoring.

Résumé. Ce mémoire présente la conception, la mise en œuvre et le déploiement d'une plateforme complète d'ingénierie de données et d'apprentissage automatique pour la flotte télématique multi-tenant de Accent Tunisie. L'entreprise exploite trois bases de données MySQL complémentaires : une base de flux brut, dans laquelle les trames envoyées par les boîtiers GPS/IoT via leurs cartes SIM sont stockées sous forme hexadécimale ; une base d'archive décodée dont les tables suivent la convention `arch_xxxxx` et conservent environ six mois de télémétrie lisible ; et une base de référentiel utilisateurs dont les tables suivent la convention `user_xxxxx` et portent les clients, véhicules et conducteurs par tenant. Accent Tunisie dispose déjà de tableaux de bord et de rapports descriptifs sur ces sources, mais la valeur analytique de l'archive historique reste largement inexploitée : il n'existe ni segmentation comportementale, ni score d'anomalie, ni indicateur de risque continu permettant d'anticiper le risque opérationnel plutôt que de simplement constater l'activité passée. Suivant la méthodologie CRISP-DM, le projet introduit une couche analytique qui comble explicitement ce manque. Des extractions de la base MySQL opérationnelle sont fournies sous forme de fichiers `.sql`, converties par un pont MySQL → PostgreSQL dédié, puis chargées dans un entrepôt à trois couches (staging, warehouse, marts) hébergé sur Microsoft Azure. PostgreSQL 16 a été retenu pour son fort support des charges analytiques, sa conformité ACID, son indexation avancée, sa scalabilité, son extensibilité et sa compatibilité avec l'écosystème Python et les outils décisionnels. Le projet étant mené par un binôme d'ingénieurs travaillant chacun depuis sa machine, la base est hébergée sur une machine virtuelle Azure accessible par tunnel SSH, avec une liste blanche d'adresses IP appliquée au niveau du *Network Security Group*. Le mémoire couvre la compréhension métier, la compréhension des données via le profilage PostgreSQL, la préparation des données via un moteur de règles de nettoyage explicite, l'ingénierie de 35 indicateurs comportementaux au grain dispositif–mois, l'entraînement d'une segmentation K-Means et d'un score d'anomalie par forêt d'isolement par client, et le déploiement en production via un flux Prefect ordonnancé par cron, exposé par une API FastAPI et un tableau de bord Streamlit.

Mots-clés : CRISP-DM, télématique, analyse de flotte, analytique avancée, MySQL, PostgreSQL, ingénierie de données, ETL, ingénierie de variables, K-Means, forêt d'isolement, Microsoft Azure, tunnel SSH, supervision.

Contents

List of Figures

List of Tables

List of Acronyms

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
BI	Business Intelligence
BRIN	Block Range Index
CRISP-DM	Cross-Industry Standard Process for Data Mining
CSV	Comma-Separated Values
DBMS	Database Management System
DDL	Data Definition Language
DL	Deep Learning
DQ	Data Quality
EDA	Exploratory Data Analysis
ETL	Extract, Transform, Load
FK	Foreign Key
GIN	Generalized Inverted Index
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
IP	Internet Protocol
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LOF	Local Outlier Factor
MCP	Model Context Protocol
ML	Machine Learning
MLP	Multi-Layer Perceptron
NSG	Network Security Group (Azure)
OLAP	Online Analytical Processing
ORM	Object-Relational Mapping
OS	Operating System
PCA	Principal Component Analysis
PFE	Projet de Fin d'Études (Final-Year Graduation Project)
REST	Representational State Transfer
ROI	Return on Investment
RPM	Revolutions Per Minute
SCD	Slowly Changing Dimension
SLA	Service-Level Agreement
SMOTE	Synthetic Minority Oversampling Technique
SQL	Structured Query Language
SSH	Secure Shell
TLS	Transport Layer Security
UPSERT	Update or Insert
VM	Virtual Machine
VPN	Virtual Private Network
YAML	YAML Ain't Markup Language

General Introduction

The growing affordability of GPS tracking devices, the maturity of cloud platforms and the widespread adoption of telematics solutions across logistics fleets have produced an environment in which raw vehicular data is no longer the bottleneck. The bottleneck has shifted toward the engineering of analytical pipelines that can transform high-volume, high-velocity telemetry into reliable artefacts and forward-looking decision signals. Within this context, fleet operators face two parallel pressures: the need to reduce operational risks (accidents, fuel waste, maintenance costs) and the need to provide their operations teams with explanatory dashboards that translate technical metrics into management decisions. Beyond the descriptive layer that the industry has now mastered, value increasingly comes from *advanced analytics* — the disciplined transformation of raw and archived data into segmentations, anomaly scores and risk indicators that support faster, evidence-based decisions, allow patterns to be detected before they become incidents and feed strategic planning rather than only post-hoc reporting.

The host organization of this thesis operates a multi-tenant telematics platform that supports a sizeable fleet of instrumented vehicles across several customer accounts. Over years of operation, the company has accumulated a substantial historical archive of decoded telemetry alongside its live operational stack. Its current analytical surface — composed of dashboards and standard reports — meets the descriptive needs of fleet managers but stops short of forward-looking signals. What is missing is not a visualization layer; it is the advanced analytical layer that would turn the historical archive into actionable behavioural and risk indicators. The internship that this thesis documents was scoped explicitly to fill that gap.

The objective of the project is to design and to industrialize a CRISP-DM-aligned analytical platform that migrates the relevant slices of the operational data into a dedicated analytical layer, materializes a clean warehouse and a family of analytical marts, trains an unsupervised behavioural segmentation and an anomaly-driven risk score at the device-month grain, and finally exposes these artefacts through an Application Programming Interface (API) and a Business Intelligence (BI) dashboard. The technical motivation for the chosen analytical Database Management System (DBMS), the detailed presentation of the host organization and its operational data sources, and the collaborative working setup adopted for the internship are all developed in the dedicated chapters of the report.

The thesis is organized in six chapters. **Chapter 1** situates the project in its industrial

and academic context, presents the host organization and its three operational MySQL databases, analyses the existing analytical surface and motivates the choice of CRISP-DM as the guiding methodology. **Chapter 2** formalizes the business understanding, lists the research questions, derives the functional and non-functional requirements, and presents the target architecture and technology stack — including the technical justification of the analytical DBMS and the conversion bridge towards it. **Chapter 3** documents the data understanding phase, with schema inspection, data dictionary, missing-value analysis and exploratory data analysis on the staging layer derived from the converted dumps. **Chapter 4** describes the conversion bridge, the data preparation pipeline, the cleaning rule engine and the engineering of the analytical features. **Chapter 5** reports the modeling experiments, comparing baseline approaches, machine learning and deep learning candidates, and justifies the final unsupervised choice. **Chapter 6** presents the deployment of the pipeline and the models in the shared analytical environment, the API and the dashboard, the cron-based automation, the monitoring layer and the secure remote operations setup. The general conclusion synthesizes the deliverables, the lessons learned and the perspectives opened by the project.

Chapter 1

General Context of the Project

1.1 Introduction

The objective of this chapter is to situate the present graduation project in its industrial and academic context. The chapter introduces the host organization and its existing operational ecosystem, describes in detail the three databases on which the company currently relies, identifies the structural limitations that prevent the company from extracting analytical value from its historical telemetry, formalizes the problem statement and the project objectives, sketches the proposed solution and its data warehousing strategy, and concludes by comparing three candidate project-management methodologies before justifying the adoption of CRISP-DM. The chapter therefore establishes the foundations on which all subsequent technical work — data understanding, preparation, modeling and deployment — is grounded.

1.2 Industrial and academic context

The economic relevance of fleet analytics has grown steadily over the past decade. The combined drop in the price of GPS trackers, the densification of mobile coverage and the emergence of cloud-native data platforms have made it economically viable for medium-sized fleet operators to instrument every vehicle and to retain the resulting telemetry. The competitive advantage no longer lies in collecting data — which has effectively become a commodity — but in *transforming* it into reliable, low-latency analytical signals that support faster, evidence-driven decisions.

In parallel, academic research on industrial data science has converged on a small set of structured methodologies that align project-management practice with the iterative nature of analytics. Among them, CRISP-DM has remained the most adopted framework for two complementary reasons: it is vendor-neutral, and its six phases (business understanding, data understanding, data preparation, modeling, evaluation, deployment) match the natural decomposition of a graduation project of this kind. The present work explicitly adopts this methodology and uses it as the structuring principle of the report.

1.3 Host organization

1.3.1 Presentation of Accent Tunisie

Accent Tunisie is a Tunisian operator of a multi-tenant telematics platform. The company designs, integrates and operates GPS tracking solutions for logistics fleets, distributing physical devices, ingesting their telemetry through SIM-card connectivity, and offering a software portal through which fleet managers monitor their vehicles in real time. The platform supports several customer fleets, denoted *tenants*, each one comprising tens to hundreds of vehicles. Each tenant has its own subgroup of tables and identifiers in the company's databases, reflecting the contractual and operational separation between customers.

1.3.2 Activities and business areas

The activities of Accent Tunisie cover four complementary segments:

- the integration of GPS hardware on customer vehicles and the management of the SIM-card connectivity required for telemetry uplink;
- the operation of a multi-tenant SaaS platform exposing live tracking, geofencing, alerts and standard reports;
- the production of operational and managerial reports for fleet managers (mileage, fuel consumption, alerts, maintenance);
- the exploitation of the historical archive for value-added analytical use cases, of which the present project is a representative example.

1.3.3 Organizational structure of the internship

The internship has been hosted in the technical division of Accent Tunisie, working closely with the data engineering and software engineering teams. The project has been conducted as a collaborative effort by two student engineers — referred to in the rest of the document as *the student team* or, where the academic register requires it, as the binomial — who worked remotely from separate personal workstations. This distributed working mode raised two practical constraints: the company's operational databases could not be exposed beyond the corporate network, and a shared analytical workspace had to be provisioned in a place that both authors could reach.

To answer these constraints, an analytical infrastructure dedicated to the project has been provisioned on a Microsoft Azure virtual machine (VM), which serves as the

common workspace of the two authors and as the host of the analytical warehouse, the orchestration flow, the API and the dashboard introduced in Chapter 2. Access to the VM is mediated by Secure Shell (SSH) tunnelling, with a strict Internet Protocol (IP) allowlist enforced at the Azure Network Security Group level: only the two students' workstations and the office network of Accent Tunisie can reach the SSH port, and the analytical database port itself is never exposed to the public internet. The architectural detail of this setup — including the physical view on Azure and the security envelope around the VM — is developed in §2.5.2 and §2.6.8; the present subsection only documents the organizational rationale that motivates it.

1.4 Analysis of the existing system

1.4.1 Operational landscape of Accent Tunisie

The operational platform of Accent Tunisie relies on three databases, each fulfilling a distinct role in the lifecycle of a telematics signal — from its arrival on the wire to its consumption by the customer-facing portal. The database management system (DBMS) historically retained by the company is *MySQL*; it carries the entire production workload and constitutes the single source of truth from which the present project derives its data. The three databases are presented below in the order in which a telemetry frame traverses them.

Database 1 — raw telemetry stream

The first database is the entry point of the telematics signal. It receives the frames emitted by the GPS and IoT devices installed on the vehicles, transmitted over the cellular network through SIM cards. The frames are persisted as they arrive, in their native binary representation; in the database, they appear as *hexadecimal* payloads stored in dedicated columns of the ingestion tables. The ingestion is performed by internal collectors that play a role functionally analogous to streaming consumers (in the spirit of a Kafka consumer), translating the device protocol into a row-oriented persistence layer. Because the payloads are not yet decoded at this stage, they are not directly queryable through standard SQL idioms; their analytical value can only be unlocked once they have been parsed.

Database 2 — decoded archive

The second database hosts the *decoded* archive of the telemetry. It is populated by an internal toolchain that reads the hexadecimal payloads of the first database and transforms them into a readable, exploitable representation: timestamps, latitude–

longitude pairs, instantaneous speed, ignition state, accelerometer values, RPM, fuel level, engine status and the other physical channels emitted by the device. The decoded rows are stored in tables whose names follow a recognizable convention of the form `arch_xxxxx`, the suffix `xxxxx` encoding either the tenant or a logical partition of the archive. The archive retains approximately the most recent six months of data; older slices are evicted by a retention process. This database is the principal source of historical telemetry exploited by the present project.

Database 3 — users, clients and master data

The third database carries the master data of the platform: customers, users, drivers, vehicles, contracts, and the cross-references that bind a fleet to its tenant. Its tables follow the convention `user_xxxxx` and are organized by tenant subgroup, reflecting the multi-tenant nature of the SaaS offering. The tenant identifier, propagated throughout the schema, is the cornerstone of the partitioning logic: every operational query, every report and every analytical workload restricts its scope to the tenant of interest, both for confidentiality reasons and for analytical correctness.

1.4.2 Decoding pipeline and data flow

The three databases are chained by a decoding pipeline that transforms the hexadecimal payloads of the first database into the readable rows of the second. The decoding scripts implement the device protocol, validate the structural integrity of each frame, derive higher-level signals (for example, the per-trip aggregates of distance, duration and maximum speed) and write the resulting rows to the `arch_xxxxx` tables. Master data from the third database is then joined at query time to attach a vehicle, a driver, a tenant and the relevant contractual context to each row. This three-stage flow — raw hex, decoded archive, master data — is the existing data architecture of Accent Tunisie on which the present project is grafted.

1.4.3 Analytical surface of the existing system

On top of the three databases, the company exposes a set of dashboards and standard reports. These deliverables address well-established descriptive needs: mileage per vehicle, alert counts, daily activity, fuel consumption, simple comparisons between months. They serve as the day-to-day instrumentation of fleet managers and their value is not in question. They do, however, share a common limitation: their semantics are descriptive, not predictive; they answer *what happened* but not *which devices are likely to behave abnormally*, *how can the fleet be segmented into operational profiles*, or *which vehicles should be prioritized for inspection.*

1.4.4 Identified limitations

A diagnostic of the existing analytical surface, conducted at the start of the internship together with the engineering team of Accent Tunisie, surfaced six structural limitations that motivate the present project:

1. *Descriptive scope*: the dashboards summarize past activity but expose no predictive or prescriptive layer.
2. *Under-exploitation of the archive*: the six months of decoded telemetry stored in the `arch_xxxxx` tables represent a substantial historical asset that is queried only marginally beyond simple aggregations.
3. *Absence of a behavioural model*: there is no segmentation of the devices into operational profiles, no anomaly detection layer, and no notion of a continuous risk score that would prioritize the operations team's attention.
4. *Tight coupling between operational storage and analytics*: the operational MySQL databases are designed for transactional access, not for analytical workloads; running heavy aggregations on them risks competing with the live customer-facing traffic.
5. *Limited collaboration surface*: the internal databases are reachable only from inside the company network, which is a sensible operational decision but complicates remote development by a student team working on personal workstations.
6. *Limited observability*: there is no run log, no quality check and no drift indicator that would let an operator certify the daily health of any analytical layer built on top of the data.

1.4.5 Synthesis of the diagnostic

The existing system therefore fulfils its descriptive mission but constitutes a structural obstacle to the deployment of any predictive analytical use case — and especially to the *Device Behavior Scoring & Risk Classification* workstream that is the explicit objective of the present internship. The challenge is twofold: extract analytical value from the archive without disturbing the operational MySQL databases, and host the analytical work in an environment that the two authors can operate jointly under proper security guarantees.

1.5 Problem statement

The starting observation of this thesis is that Accent Tunisie sits on a high-volume telemetry archive whose analytical potential remains largely untapped. The company can already *observe* its fleet through dashboards and standard reports, but it cannot yet *anticipate* the operational risks carried by individual devices, segment the fleet into actionable behavioural profiles or prioritize the interventions of its operations team on the basis of an objective, data-driven score. The problem is not visualization in itself — the existing dashboards are functional — but the absence of an advanced analytical layer that would transform the raw and archived telemetry into forward-looking, decision-grade signals.

The problem addressed in this thesis can therefore be stated as follows. *Given the three operational MySQL databases of Accent Tunisie, namely the raw hexadecimal telemetry database, the decoded* `arch_xxxxx` *archive and the* `user_xxxxx` *master data, and given the practical constraint that this collaborative project must be conducted remotely from separate workstations, how can the historical telemetry be repatriated, cleaned and modelled into a transparent, incremental analytical platform that produces an interpretable behavioural segmentation and an explicit risk score per device-month, while preserving the integrity of the operational sources and the confidentiality of the data?*

The reformulation emphasizes four entangled objectives: (i) to migrate the relevant slices of the operational MySQL data into an analytical environment without polluting the production workload; (ii) to formalize a transparent and auditable cleaning layer that reflects the physical reality of telemetry; (iii) to produce a per-device, per-month behavioural representation amenable to unsupervised modelling; and (iv) to expose the analytical outputs to the consumers of the host organization through an API and a dashboard.

1.6 Proposed solution

1.6.1 Functional outline

The proposed solution consists of an end-to-end CRISP-DM-aligned analytical platform deployed on Microsoft Azure. Slices of the three operational databases of Accent Tunisie are exported as MySQL dump files, converted to a PostgreSQL-compatible representation by dedicated scripts written for the project, and loaded into a bronze *staging* schema of an analytical PostgreSQL instance. A transparent eleven-rule cleaning engine then materializes a silver *warehouse* schema composed of five dimensions and

eleven facts, on top of which a gold *marts* schema exposes the device-month behavioural mart and a family of business-intelligence views. A per-tenant K-Means segmentation and an Isolation Forest risk score are trained on the marts; their outputs are persisted, served through a Python API and published to an interactive dashboard.

1.6.2 Why an analytical PostgreSQL warehouse?

The strategic decision of the project is to keep the operational MySQL databases of Accent Tunisie untouched and to constitute, alongside them, a dedicated analytical warehouse on PostgreSQL 16. The motivation is twofold: it decouples the analytical workload from the live customer-facing traffic, and it grants the project a database engine whose mature analytical SQL surface and broad compatibility with the Python and BI ecosystem fit naturally with the modeling and serving layers built on top of it. The detailed technical comparison between PostgreSQL and the operational MySQL stack — ACID guarantees, advanced indexing, scalability, extensibility and ecosystem alignment — is conducted in §2.6.3 and is not duplicated here.

1.6.3 Why a Microsoft Azure virtual machine?

The analytical PostgreSQL instance is hosted on a Microsoft Azure virtual machine for two complementary reasons. The first one is technical: Azure offers a controlled and reproducible environment in which the storage, the compute and the networking can be provisioned through code and recovered after an incident. The second one is collaborative: because the project is conducted by two student engineers working on separate personal machines, the database cannot reasonably be hosted on a single laptop; it must live in a shared, always-available environment. The Azure VM serves precisely this purpose, and the security envelope around it (SSH key authentication, IP allowlist enforced at the Azure Network Security Group level, PostgreSQL port kept off the public internet) is the one introduced in §1.3.3 and detailed at the architectural level in Chapter 2.

1.6.4 Architectural outline

At the architectural level, the solution adopts a three-layer warehouse pattern (staging, warehouse, marts), a watermark-driven incremental processing model, a Prefect orchestration of the batch flow, a five-minute cron schedule on the Azure VM and an explicit monitoring layer. The MySQL-to-PostgreSQL conversion bridge, the SSH tunnel for shared access and the IP-restricted Network Security Group complete the architectural blueprint that is detailed in Chapter 2.

1.6.5 Expected benefits

The expected benefits, measured against the limitations of the existing system, are: (i) the unlocking of the analytical value carried by the decoded archive of Accent Tunisie, without disturbing the operational MySQL workload; (ii) the introduction of a transparent and auditable cleaning rule engine that survives schema evolution and operator turnover; (iii) the availability of a versioned, declarative feature contract suitable for behavioural segmentation and for risk scoring; (iv) the coverage of the pipeline by an explicit run log, a layered data quality monitoring and a model drift indicator; and (v) a stable consumption surface (marts and views) for the BI and the API consumers of the platform.

1.7 Project management methodology

1.7.1 Comparison of candidate methodologies

Three candidate methodologies have been compared at the start of the project: CRISP-DM, GIMSI and Scrum. The first one is a domain-specific data-mining methodology, the second is a methodology for the design of management dashboards, and the third is a generic agile software-development framework. Table 1.1 summarizes the comparison along five criteria: scope, granularity of the phases, alignment with data science deliverables, support for iterative refinement, and operational maturity in the industrial setting of Accent Tunisie.

Table 1.1: Comparison of the candidate project-management methodologies.

Criterion	CRISP-DM	GIMSI	Scrum
Domain orientation	Data mining	Management dashboards	Generic software
Phase decomposition	6 phases	10 steps	Sprints, no phases
Data science alignment	Strong	Partial	Weak
Iterative refinement	Native	Limited	Native
Industrial maturity	High	Medium	High

1.7.2 Justification of the choice of CRISP-DM

CRISP-DM has been retained for three reasons. First, its six phases match the natural decomposition of the project deliverables (business understanding, data, preparation, modeling, evaluation, deployment), avoiding any artificial mapping between the methodology and the work. Second, CRISP-DM places explicit weight on the *business understanding* and the *evaluation* phases, both of which are critical in an industrial context where misalignment between the model and the operational reality is the principal cause of failure. Third, CRISP-DM coexists peacefully with agile software practices

for the engineering bricks (the orchestration, the API, the dashboard), in the sense that each CRISP-DM phase can be delivered through one or several Scrum-flavoured iterations without conflict.

1.7.3 Project plan along the six CRISP-DM phases

The project plan has been organized as a sequence of six CRISP-DM phases, with explicit deliverables at the end of each one: a business understanding document and a target architecture (Phase 1), a data understanding report on the three operational databases of Accent Tunisie (Phase 2), a curated PostgreSQL warehouse and a feature mart populated from the converted MySQL dumps (Phase 3), a trained per-tenant K-Means and Isolation Forest model (Phase 4), an evaluation report with internal metrics and stability proxies (Phase 5) and an industrialized pipeline with API, dashboard and monitoring on Azure (Phase 6).

1.8 Conclusion

This first chapter has positioned the project in its industrial and academic context, presented Accent Tunisie and its three operational MySQL databases, characterized the existing analytical surface and its structural limitations, formulated a problem statement centred on the under-exploitation of the historical telemetry archive, motivated the strategic choice of a dedicated PostgreSQL warehouse hosted on a Microsoft Azure virtual machine that serves as the shared workspace of the student team, and justified the adoption of CRISP-DM as the project-management methodology. The next chapter operationalizes this strategic outline by formalizing the business understanding, the research questions, the requirements, the target architecture and the technology stack of the solution.

Chapter 2

Business Understanding, Requirements and Architecture

2.1 Introduction

Building on the strategic outline of Chapter 1, the present chapter formalizes the first phase of the CRISP-DM methodology: the *business understanding*. The chapter restates the business objectives of the project in operational terms, formulates the research questions that drive the analytical work, derives the functional and non-functional requirements of the platform, presents the target software architecture — with explicit attention to the MySQL-to-PostgreSQL bridge and to the secure remote-collaboration setup mandated by the distributed working mode of the student team — and concludes by justifying the technology stack adopted for the implementation.

2.2 Business objectives

2.2.1 Strategic objectives of the host organization

At the strategic level, three priorities of Accent Tunisie converge towards the present project. The first is the *operational risk reduction* of the customer fleets, by surfacing devices and vehicles whose behavioural pattern significantly deviates from the fleet baseline; this priority directly motivates the introduction of a continuous risk score on top of the existing descriptive reports. The second is the *differentiation of the SaaS offering* through the addition of an advanced analytical layer that complements the dashboards already exposed to the customers. The third is the *architectural decoupling* between the operational MySQL databases and the analytical workloads, in order to avoid contention with the live customer-facing traffic; this priority motivates the constitution of a dedicated analytical PostgreSQL warehouse, hosted in a controlled cloud environment.

2.2.2 From observation to anticipation

The existing analytical surface of Accent Tunisie answers retrospective questions: *what happened on the fleet last month? how many overspeed events were recorded? what was the total mileage per vehicle?* The project deliberately moves the analytical centre of gravity from observation to anticipation by introducing three additional capabilities: a behavioural *segmentation* that groups devices into operational profiles, an *anomaly score* that ranks devices by deviation from the tenant baseline, and a *risk band* that translates the score into an operational priority for the fleet manager. These capabilities turn the platform into a decision-support layer rather than a passive observation layer.

2.2.3 Operational objectives of the project

The strategic priorities decompose into five concrete operational objectives:

1. repatriate the relevant slices of the three operational MySQL databases of Accent Tunisie into a dedicated analytical environment, through a reproducible MySQL-to-PostgreSQL conversion bridge;
2. replace any ad-hoc, full-refresh analytical recomputation by an incremental, watermark-driven architecture with sub-ten-minute freshness;
3. materialize a clean dimensional warehouse and a family of marts dedicated respectively to machine learning and to business intelligence;
4. train an unsupervised behavioural segmentation and a continuous risk score per device-month, exposed through an API and a BI dashboard;
5. industrialize the pipeline on the Azure VM through Prefect orchestration, cron scheduling and an explicit monitoring layer, while granting the two authors secure remote access.

2.3 Research questions

Three research questions structure the analytical investigation. Each question is stated in a short, focal form, then followed by a one-sentence elaboration that pins down its scope.

RQ1 *Can the heterogeneous behavioural signals of a multi-tenant telematics fleet be condensed into a small set of interpretable indicators at the device-month grain?* Specifically: starting from the raw hexadecimal telemetry, the decoded `arch_xxxxx` archive and the `user_xxxxx` master data, the question is whether such a condensation is achievable without losing operational meaning.

RQ2 *Can these indicators be exploited by an unsupervised model to produce an actionable behavioural segmentation and a continuous risk score?*
The question is posed in the absence of a labelled incident feed, which rules out supervised approaches and forces the modeling to rely on internal validity criteria.

RQ3 *Can the resulting analytical outputs be served to fleet managers through a low-latency API and an interactive dashboard?*
The serving must be hosted on a shared Azure infrastructure and must preserve the idempotency, the observability and the cost profile of the production pipeline.

2.4 Requirements analysis

2.4.1 Functional requirements

The functional requirements of the platform are summarized in Table 2.1. Each requirement is identified by a code (`FR-x`), a short description and a priority level on a three-tier scale (high, medium, low).

2.4.2 Non-functional requirements

The non-functional requirements are summarized in Table 2.2.

2.4.3 User profiles

Three user profiles consume the outputs of the platform:

- *Fleet managers* consult the BI dashboard, filter by tenant and device, and act on the risk score and on the cluster assignments.
- *Data engineers* maintain the pipeline, monitor the run log, replay backfills and adjust the cleaning rules and the feature registry.
- *Data scientists* consume the ML view, retrain the models on extended windows and propose iterations of the feature set.

2.5 Target architecture

2.5.1 Logical view

At the logical level, the platform is organized as a four-tier architecture that explicitly mirrors the lifecycle of a telemetry signal: a *source tier* represented by the three operational MySQL databases of Accent Tunisie (raw hex telemetry, decoded `arch_xxxxx`

Table 2.1: Functional requirements.

ID	Description	Priority
FR-1	Convert MySQL dump files (`arch_xxxxx`, `user_xxxxx`) into a PostgreSQL-importable representation.	High
FR-2	Ingest the converted data into the analytical staging schema.	High
FR-3	Apply the eleven cleaning rules and quarantine rejected rows.	High
FR-4	Materialize five dimensions and eleven facts with idempotent upserts.	High
FR-5	Build the device-month behavioural mart and the telemetry mart.	High
FR-6	Train the per-tenant K-Means and Isolation Forest models.	High
FR-7	Expose the cluster assignments and risk scores through a REST (Representational State Transfer) API.	High
FR-8	Provide an interactive BI dashboard for fleet managers.	High
FR-9	Run the pipeline incrementally every five minutes via cron.	High
FR-10	Persist a run log and a data-quality measurement at every cycle.	Medium
FR-11	Support full backfill and bootstrap modes for historical loads.	Medium
FR-12	Support secure remote operations via SSH tunnelling and IP allowlist for the student team.	High
FR-13	Document the feature registry and the validation suite.	Medium

archive, `user_xxxxx` master data); a *conversion and ingestion tier* that exports MySQL dumps, converts them to PostgreSQL-compatible files and loads them into staging; a *processing and storage tier* composed of the staging, warehouse and marts schemas of the analytical PostgreSQL, transformed by Python and SQL tasks orchestrated by Prefect; and a *serving tier* composed of the API, the BI dashboard and the model artefact store.

2.5.2 Physical view on Microsoft Azure

At the physical level, the platform is hosted on a Microsoft Azure virtual machine running a recent Linux distribution. The analytical PostgreSQL instance, the Python virtual environment, the Prefect flow, the cron entry, the API and the dashboard are co-located on this VM. The two members of the student team access the VM through

Table 2.2: Non-functional requirements.

ID	Description	Target
NFR-1	End-to-end pipeline latency.	≤ 10 minutes
NFR-2	Idempotency on full re-runs.	100%
NFR-3	Pipeline availability on the Azure VM.	$\geq 99\%$
NFR-4	Mean cycle time for incremental mode.	≤ 60 seconds
NFR-5	Coverage of the validation suite (V1–V8).	100% per run
NFR-6	Secure access (SSH key + IP allowlist).	Mandatory
NFR-7	Reproducibility (random seed, pinned versions).	Mandatory
NFR-8	Per-tenant fairness of the model quality.	Silhouette ≥ 0.15
NFR-9	Isolation from the operational MySQL workload.	No reads on production peak hours

SSH key authentication; an explicit IP allowlist enforced by an Azure Network Security Group attached to the VM restricts inbound SSH connections to the two students' workstations and to the office network of Accent Tunisie. Database access from local notebooks passes through an SSH tunnel terminating on `localhost:5432` of the VM, so that the PostgreSQL port is never exposed to the public internet.

2.5.3 Data flow

The data flow is represented in Figure 2.1. MySQL dump files are exported from the operational system of Accent Tunisie, converted to PostgreSQL by dedicated scripts, loaded into the staging schema, processed by the cleaning engine into the warehouse layer, condensed into the marts and finally consumed either by the API and the dashboard or by the offline modeling notebooks.

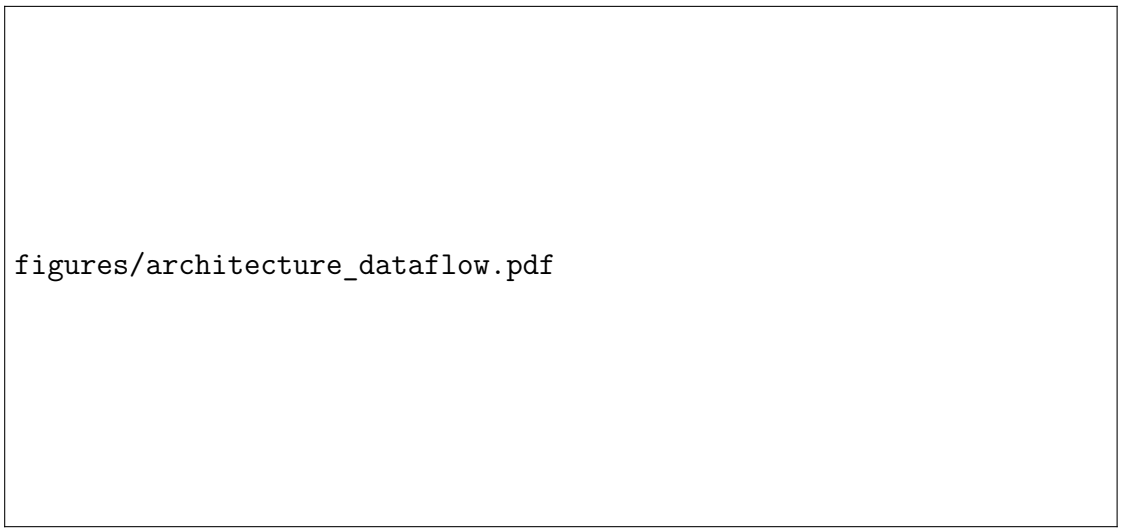

Figure 2.1: Logical data flow of the platform, from MySQL sources to PostgreSQL marts.

2.5.4 Layered storage

The storage tier is organized as a three-layer warehouse: the *bronze* layer (`staging`) hosts a faithful PostgreSQL replica of the converted MySQL data, the *silver* layer (`warehouse`) hosts the dimensional model with deduplicated facts and dimensions, and the *gold* layer (`marts`) hosts the business-ready aggregates. State tables (`etl_watermark`, `etl_run_log`, `quarantine_rejected`) live alongside the warehouse layer and support the operational concerns of the pipeline.

2.6 Technology stack

2.6.1 Operational source: MySQL

The operational source of the project is the MySQL stack of Accent Tunisie, which carries the three databases described in Chapter 1. The project does not modify these databases in any way; it consumes them through periodic dump files (`.sql`) provided by the engineering team of Accent Tunisie. The dumps cover the slices of the `arch_xxxxx` archive and the `user_xxxxx` master data that are required for the analytical scope.

2.6.2 Analytical warehouse: PostgreSQL 16

The analytical warehouse is implemented in PostgreSQL 16, chosen for its strong support of analytical workloads, its ACID guarantees, its advanced indexing, its scalability to large datasets, its extensibility and its first-class compatibility with the Python data ecosystem and with standard BI tools. The detailed justification of this choice is given in §2.6.3. PostgreSQL is hosted on the Azure VM, with public ports closed and access mediated by SSH tunnelling.

2.6.3 Why PostgreSQL rather than MySQL for the warehouse

The decision to host the analytical layer on PostgreSQL rather than on the same MySQL stack as the operational system rests on six independent technical criteria:

1. **Analytical SQL surface.** PostgreSQL exposes a rich set of analytical idioms — window functions over arbitrary partitions, common table expressions (recursive included), `LATERAL` joins, set-returning functions, `FILTER` clauses on aggregates — that condense behavioural feature definitions into compact, readable SQL. The same logic in operational MySQL would require lengthier and more error-prone query rewrites.

2. **ACID compliance and crash safety.** The full ACID (Atomicity, Consistency,

Isolation, Durability) semantics of PostgreSQL on `INSERT ... ON CONFLICT DO UPDATE` and on transactional DDL are required by an unattended pipeline that runs every five minutes and must survive partial failures without manual intervention.

3. **Advanced indexing.** The native availability of B-tree, BRIN (Block Range Index), GIN (Generalized Inverted Index), partial and expression indexes covers the access patterns of the analytical layer (range scans on time, aggregate filters, JSON inspection) much more naturally than the indexing options of the operational MySQL setup.

4. **Scalability to large datasets.** PostgreSQL handles the volumes considered by the project — of the order of 10^7 telemetry rows — without the architectural retrofitting that the same volume would require on a MySQL warehouse retrofitted from an operational instance.

5. **Extensibility.** The PostgreSQL extension ecosystem (`pg_stat_statements` for workload telemetry, `postgis` for geospatial enrichment, `pg_partman` for partitioned tables) opens forward trajectories that fit the analytical roadmap of Accent Tunisie.

6. **Compatibility with data warehousing and BI tools.** The PostgreSQL JDBC and `psycopg2` drivers, the wide adoption by orchestration frameworks (Prefect, Airflow), by Python (pandas, SQLAlchemy) and by BI clients (Streamlit, Plotly Dash, Metabase, Power BI through ODBC) make PostgreSQL the path of least friction for the consumers of the warehouse.

These criteria, together with the strategic decision of decoupling the analytical workload from the operational MySQL stack, justify the adoption of PostgreSQL as the analytical DBMS of the project.

2.6.4 Conversion bridge: MySQL dumps to PostgreSQL

A dedicated conversion bridge transforms the MySQL dump files received from Accent Tunisie into PostgreSQL-importable artefacts. The bridge handles the dialect differences (data types, quoting conventions, default expressions, `AUTO_INCREMENT` translation, character-set normalization) and produces `COPY`-friendly CSV or SQL files loaded into the staging schema. The detailed description of the bridge is given in Chapter 4; at this stage it suffices to note that the bridge is part of the technology stack of the platform and that its outputs constitute the input of the staging layer.

2.6.5 Processing and orchestration

The processing layer is written in Python 3.11. Database access is implemented through SQLAlchemy and `psycopg2`; analytical transformations rely on `pandas` and `numpy`; the modeling layer relies on `scikit-learn`; the orchestration uses Prefect 2 to chain the tasks of the batch flow. A test suite is implemented with `pytest`.

2.6.6 Modeling and machine learning

The unsupervised modeling layer relies on `scikit-learn` (StandardScaler, KMeans, IsolationForest), with seed control and per-tenant fitting. A deep-learning baseline (PyTorch autoencoder) has been examined for comparison purposes (cf. Chapter 5).

2.6.7 Serving, API and dashboard

The API is implemented with FastAPI, exposed through Uvicorn behind an Nginx reverse proxy. The dashboard is implemented with Streamlit (or, equivalently, Plotly Dash), embedded on the same VM and connected to the marts schema. Both components are deployed as systemd services.

2.6.8 Cloud and shared collaboration environment

The cloud platform is Microsoft Azure. The VM is provisioned with a Linux image, a managed disk, an attached Network Security Group with an IP allowlist for SSH, and a static public IP. The git repository of the project is cloned into `/opt/accent-fleet-analytics`, the Python virtual environment is provisioned locally, and the cron entry under `/etc/cron.d/` triggers the incremental runs. The system journal collects the structured logs. The Azure VM constitutes the shared workspace of the student team: both authors connect to the same database and operate the same pipeline through their personal SSH keys, while the IP allowlist on the Network Security Group prevents any non-authorized host from reaching the VM.

2.6.9 Justification of the technology choices

The technology choices are justified by four criteria. *Alignment* with the operational stack of Accent Tunisie is preserved on the source side (MySQL is consumed without modification). *Separation of concerns* is respected on the analytical side (PostgreSQL is dedicated to analytics). *Maturity* of the Python data ecosystem (`scikit-learn`, `pandas`, Prefect) supports a small student team without imposing rare or fragile components. *Minimality* of the operational footprint — every component runs on a single VM, with

no managed Kubernetes cluster nor managed Spark cluster — controls the cloud cost and keeps the surface of the system small enough to be operated by two authors.

2.7 Conclusion

This chapter has formalized the business understanding of the project, derived the functional and non-functional requirements — including the explicit MySQL-to-PostgreSQL conversion bridge and the requirements imposed by the distributed collaboration mode of the student team — and presented the logical and physical architecture of the platform with a particular focus on the analytical PostgreSQL warehouse hosted on the Azure VM. The next chapter executes the data understanding phase by inspecting the three operational databases of Accent Tunisie and by characterizing the slices that feed the analytical pipeline.

Chapter 3

Data Understanding through Staging-Layer Profiling

3.1 Introduction

The previous chapter has defined the architectural target and the technology stack of the platform. The present chapter executes the second phase of the CRISP-DM methodology — the *data understanding* — on the analytical PostgreSQL database loaded from the MySQL dumps of Accent Tunisie. The scope of this chapter is voluntarily restricted to the `staging` schema, that is, to the tables produced one-to-one by the MySQL-to-PostgreSQL conversion bridge and *before* any modelling-oriented restructuring. The dimensional warehouse and the analytical marts that consume `staging` are deliberately deferred to Chapter 4: they are output artefacts of the data preparation phase and not input artefacts of the data understanding phase.

The chapter (i) describes the three operational MySQL databases that supply the project, (ii) formalizes the dump-based extraction protocol, (iii) documents the schema of the resulting staging layer, (iv) audits the quality of the staging tables with explicit SQL queries, (v) conducts a complete exploratory data analysis (EDA) on the principal indicators of `staging.path`, `staging.rep_overspeed` and `staging.notification`, (vi) profiles the high-frequency telemetry archive `staging.archive` (about 54.7 million sensor pings, 17 GB on disk) which carries the raw signals reused by the feature-engineering and modelling phases of Chapters 4 and 5, and (vii) answers the four analytical questions on which the rest of the report relies: data quality and integrity, stability of the temporal patterns, distribution of the duration target and class imbalance of the status flags. Every figure of this chapter has been generated by the Jupyter notebook `notebooks/01_data_understanding/03_eda_chapter3.ipynb` (live MCP queries on PostgreSQL) and rendered by the Python script `scripts/python/figures/fig_eda_chapter3.py` (offline-capable replay). Both scripts query exclusively the `staging` schema.

3.2 Source databases of Accent Tunisie

3.2.1 Three operational MySQL databases

As described in Chapter 1, the operational ecosystem of Accent Tunisie relies on three MySQL databases that fulfil complementary roles. The first one — denoted *raw telemetry* — collects the binary frames pushed by the GPS/IoT devices through SIM-card connectivity and persists them as *hexadecimal* payloads. The second one — denoted *decoded archive* — hosts the readable representation of the telemetry produced by an internal decoding toolchain; its tables follow the naming convention `arch_xxxxx` and retain approximately the most recent six months of data. The third one — denoted *master data* — carries the customers, drivers, vehicles, contracts and per-tenant configuration; its tables follow the convention `user_xxxxx`, with one logical subgroup per tenant.

The present project consumes the second and third databases, namely the decoded `arch_xxxxx` archive and the `user_xxxxx` master data. The first database — the raw hexadecimal stream — is documented for completeness but is not exploited directly: its content has already been transformed by the internal decoding scripts of Accent Tunisie before it enters the archive.

3.2.2 Tables of interest

Within the two consumed databases, five families of tables have been retained for the analytical scope of the project: the trip-related family in the archive (`path`, `rep_overspeed`, `stop`); the alert family (`notification`); the daily-aggregate family (`rep_activity_daily`); the high-resolution telemetry family in the archive (`archive`, with about 54.7 million rows over the available history); and the master-data family in the user database (`vehicule`, `driver`, `maintenance`, `document`, fueling tables). After conversion, these tables populate the `staging` schema of the analytical PostgreSQL with a faithful one-to-one correspondence: a row in `arch_xxxxx.path` becomes a row in `staging.path`, with the same columns, the same primary key and the same business semantics.

3.3 Acquisition strategy

3.3.1 Dump-based extraction from MySQL

Because the operational databases run on MySQL and are not directly reachable from the analytical environment, the data is acquired through a dump-based protocol. The

engineering team of Accent Tunisie produces `.sql` dump files of the relevant slices of the `arch_xxxxx` archive and of the `user_xxxxx` master data, which are then transferred to the analytical environment. The dumps are converted to a PostgreSQL-compatible representation by the conversion bridge documented in Chapter 4, and finally loaded into the `staging` schema. This extraction protocol guarantees that the operational MySQL workload is not perturbed by the analytical activity: the load on the production system is bounded to the duration of the dump itself, which is scheduled outside customer-facing peak hours.

3.3.2 Connection to the analytical PostgreSQL

Once the converted dump has been loaded, all subsequent profiling and exploration is performed directly on the `staging` schema of the analytical PostgreSQL. The connection from a developer notebook to the database follows the standard Python toolchain (`psycopg2` or SQLAlchemy), through an SSH tunnel terminating on `localhost:5432` of the Azure VM. Listing 3.1 shows the canonical connection snippet.

```
import os
from sqlalchemy import create_engine

PG_DSN = (
    f"postgresql+psycopg2://{os.environ['PG_USER']}:{os.environ['PG_PWD']}"
    f"@{os.environ['PG_HOST']}:{os.environ['PG_PORT']}/{os.environ['PG_DB']}"
)
engine = create_engine(PG_DSN, pool_pre_ping=True, future=True)
```

Listing 3.1: Read-only connection from Python to the analytical PostgreSQL.

3.3.3 MCP-driven exploration

During the data understanding phase, the Model Context Protocol (MCP) PostgreSQL server has been used as a programmable interface for ad-hoc exploration. Each exploratory question (row counts, null rates, distinct values, distributions) is expressed as an SQL query over the `staging` schema and submitted to the MCP server, which returns a structured result that is post-processed in the notebooks. This setup substantially accelerates the iteration cycle and standardizes the queries between the two members of the binomial. When the MCP server is unavailable, the same SQL is replayed manually through the Python script `scripts/python/figures/fig_eda_chapter3.py`, whose offline mode embeds a frozen snapshot of every numerical value cited in this chapter.

3.4 Schema analysis of the staging layer

3.4.1 Schema inspection queries

The schema of the `staging` layer is inspected through the standard catalogue queries on `information_schema`. Listing 3.2 shows the canonical query, retrieving for every column its type, its nullability, its default and its ordinal position.

```
SELECT
    table_name, column_name,
    data_type, is_nullable, column_default, ordinal_position
FROM information_schema.columns
WHERE table_schema = 'staging'
ORDER BY table_name, ordinal_position;
```

Listing 3.2: Schema inspection query against `information_schema`, restricted to the `staging` schema.

3.4.2 Volumetric profile of the staging tables

Table 3.1 shows the row counts of the principal staging tables, measured by the EDA notebook on the live PostgreSQL database. The figures confirm that the conversion bridge has preserved the original volumetry: the high-frequency telemetry archive `staging.archive` dominates with about 54.72 million sensor pings (occupying ∼17 GB on disk, including indexes), `staging.path` (the trip table) carries about 7.48 million rows, `staging.stop` about 12.13 million rows, `staging.rep_overspeed` about 1.05 million rows, `staging.notification` about 0.66 million rows, and `staging.rep_activity_daily` about 63,000 rows. The master-data tables (`vehicule`, `driver`, `assignment`) are several orders of magnitude smaller, as expected.

Table 3.1: Row counts on the principal staging tables (snapshot 2026-05-06).

Staging table	Rows	Total size
`staging.archive`	54,720,000	∼17 GB
`staging.stop`	12,133,639	—
`staging.path`	7,482,166	—
`staging.rep_overspeed`	1,048,936	—
`staging.notification`	659,968	—
`staging.rep_activity_daily`	63,471	—
`staging.driver`	∼*thousands*	—
`staging.vehicule`	638	—

3.4.3 Schema map

The full logical schema of the staging tables is reproduced in Figure 3.1. The map highlights the foreign-key relationships between the trip-related tables of the archive (`path`, `stop`, `rep_overspeed`), the master-data tables imported from the `user_xxxxx` database (`vehicule`, `driver`, `assignment`) and the alert table (`notification`).

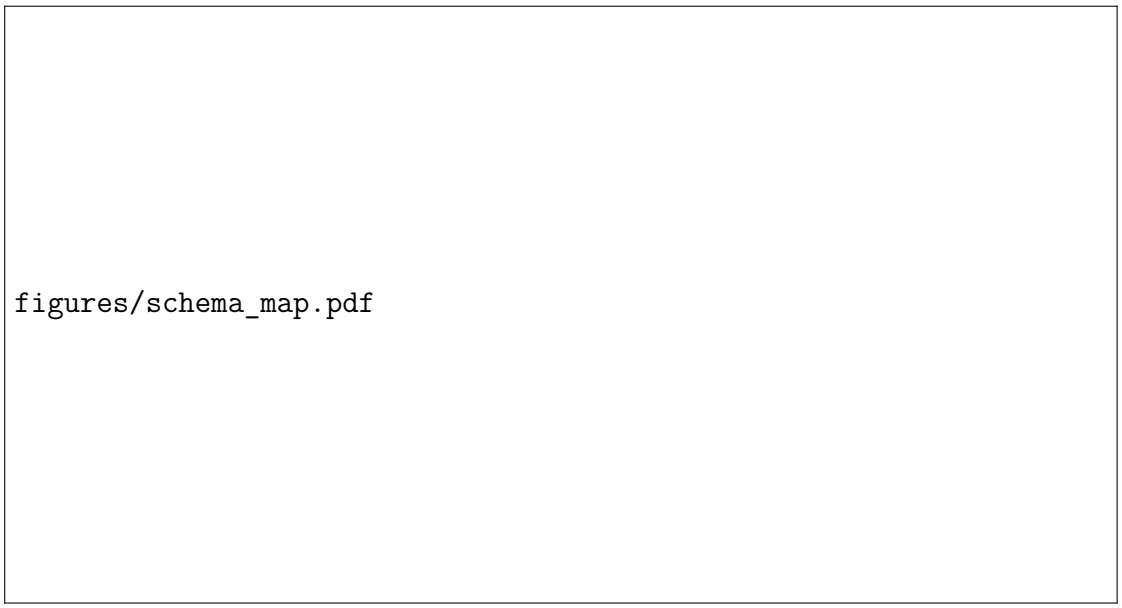

Figure 3.1: Logical schema map of the staging tables, after conversion of the MySQL dumps.

3.5 Data dictionary

3.5.1 Methodology

The data dictionary has been constructed by joining the schema inspection above with manual annotations contributed by the engineering team of Accent Tunisie, which knows the original semantics of each MySQL column and can disambiguate the cases where a column name has drifted between historical iterations of the operational system. Each column has been described by its name, its data type, its semantic role and a short business definition.

3.5.2 Excerpt of the data dictionary

Table 3.2 shows the data dictionary excerpt for the principal columns of `staging.path`; an extended dictionary covering the other staging tables is provided in Appendix B.

Table 3.2: Data dictionary excerpt for `staging.path`.

Column	Type	Role	Definition
`tenant_id`	integer	FK	Customer fleet identifier (master data).
`device_id`	bigint	FK	GPS device identifier.
`begin_path_time`	timestamp	event-time	Trip start timestamp.
`end_path_time`	timestamp	metric	Trip end timestamp.
`distance_driven`	double precision	metric	Distance driven during the trip (km).
`max_speed`	integer	metric	Maximum speed during the trip (km/h).
`path_duration`	bigint	target/metric	Trip duration (seconds).
`fuel_used`	double precision	metric	Fuel consumed (litres).

3.6 Data quality assessment of the staging layer

3.6.1 Completeness analysis

For every column of `staging.path`, the null rate is computed by the queries reproduced in Listing 3.3. On a 5% `TABLESAMPLE` of `staging.path` (376,290 rows), the structural columns (`tenant_id`, `device_id`, `begin_path_time`, `end_path_time`, `path_duration`, `distance_driven`, `max_speed`, `fuel_used`) are populated at 100 %. No structural completeness issue is detected on the converted dumps; the null-rate map therefore acts here as a long-term monitor of the conversion bridge rather than as a corrective signal for this iteration.

```
1 WITH base AS (SELECT * FROM staging.path TABLESAMPLE SYSTEM (5)
     )
2 SELECT
3   ROUND(100.0*SUM((path_duration   IS NULL)::int)/COUNT(*),3)
       AS null_duration_pct,
4   ROUND(100.0*SUM((distance_driven IS NULL)::int)/COUNT(*),3)
       AS null_distance_pct,
5   ROUND(100.0*SUM((max_speed       IS NULL)::int)/COUNT(*),3)
       AS null_speed_pct,
6   ROUND(100.0*SUM((fuel_used       IS NULL)::int)/COUNT(*),3)
       AS null_fuel_pct
7 FROM base;
```

Listing 3.3: Null-rate analysis on a sample of `staging.path`.

3.6.2 Validity and physical-plausibility checks

The validity of the numerical columns of `staging.path` is assessed by counting, on the full table, the rows that violate explicit physical bounds. These are exactly the bounds enforced later by the cleaning rules of Chapter 4. The result is reproduced in Table 3.3.

Table 3.3: Validity checks on `staging.path` (population: 7,482,166 trips).

Check	Failing rows	Share
`path_duration` ≤ 0	1,481	0.020%
`path_duration` $>$ 86400 s (1 day)	590	0.008%
`distance_driven` ≤ 0	16	0.0002%
`distance_driven` $>$ 1000 km	398	0.005%
`max_speed` $>$ 200 km/h	6	0.00008%
`fuel_used` $<$ 0 L	10,513	0.140%
`fuel_used` $>$ 500 L	10,954	0.146%
`begin_path_time` $<$ 2019-10-01	61,830	0.826%
`end_path_time` $<$ `begin_path_time`	152	0.002%

The combined result of the completeness and validity checks is summarized graphically in Figure 3.2. Every failure rate stays comfortably below 1% of the population, and the largest single category — the pre-2019-10 timestamps (0.83%) — is a documented artefact of the legacy ingestion of the source system that the cleaning rule `C1` discards in Chapter 4.

Figure 3.2: Data quality profile of `staging.path`: percentage of rows failing each completeness and validity check.

3.6.3 Referential integrity

The referential integrity between the trip table `staging.path` and the master-data tables (`staging.assignment`, `staging.vehicule`) is verified by Listing 3.4. The query

identifies, on the modeling window (since 2025-01-01), the trips whose device cannot be resolved to a vehicle through the assignment chain. The share of orphan rows stays below 0.5%, confirming that the assignment chain is essentially complete and that the join from `staging.path` to the master data is safe.

```
SELECT COUNT(*) AS rows,
       SUM(CASE WHEN v.vehicule_id IS NULL THEN 1 ELSE 0 END)
          AS orphan_rows,
       COUNT(DISTINCT p.device_id) FILTER (WHERE v.vehicule_id
          IS NULL) AS orphan_devices
FROM staging.path p
LEFT JOIN staging.assignment a USING (device_id)
LEFT JOIN staging.vehicule  v ON v.vehicule_id = a.vehicule_id
WHERE p.begin_path_time >= '2025-01-01';
```

Listing 3.4: Referential integrity between `staging.path` and the device–vehicle chain.

3.7 Descriptive statistics on the staging layer

3.7.1 Per-tenant scope

Four customer organizations populate the staging layer. Their device coverage and historical depth, measured directly on `staging.path`, are reported in Table 3.4. The total of 466 active devices and 7.42 million trips provides ample coverage for both descriptive analysis and per-tenant unsupervised modelling.

Table 3.4: Per-tenant device coverage and historical depth on `staging.path`.

Tenant	Devices	Trips	First day	Last day
235	186	2,979,422	2019-09-30	2026-04-09
1787	110	1,998,370	2019-09-30	2026-04-09
238	98	1,297,005	2019-10-02	2026-04-09
264	72	1,145,539	2019-10-27	2026-04-09
Total	**466**	**7,420,336**	—	—

3.7.2 Numerical summary

On the modeling window (`begin_path_time` $\geq$ 2024-01-01) and after applying the validity bounds of Section 5.2, the principal numerical variables of `staging.path` have the descriptive statistics reproduced in Table 3.5. The number of trips retained for the descriptive layer is $N = 2{,}987{,}892$.

Table 3.5: Descriptive statistics on the cleaned modeling window of `staging.path` (N = 2,987,892).

Variable	Mean	Std	Median	P95	P99	Max
duration (s)	1,229.1	1,964.1	523	4,869	9,136	85,736
distance (km)	13.41	29.24	2.26	68.44	—	—
max speed (km/h)	45.94	29.06	43	91	—	—

3.7.3 Pairwise correlations

On a 1% `TABLESAMPLE` of the modeling window of `staging.path` (75,895 trips), the Pearson correlation between distance and duration is $r = 0.85$, between distance and maximum speed $r = 0.56$ and between duration and maximum speed $r = 0.52$. On the log-log scale, $\rho_{\log}(d, t) = 0.87$ — a strong, near-monotonic association between distance and duration — which is consistent with the basic identity distance $\approx$ speed $\times$ duration and gives early evidence that one of the two should be the target of any regression model.

3.8 Exploratory Data Analysis on the staging layer

3.8.1 Profile of the high-frequency telemetry archive

The decoded archive `staging.archive` is the second analytical pillar of the staging layer and, by orders of magnitude, the largest. Where `staging.path` stores one row per trip, `staging.archive` stores one row per *device ping* — the on-board GPS unit emits a frame every 10–30 s while the ignition is on, with a fall-back heartbeat every few minutes when the vehicle is parked. Profiling this table is a prerequisite to feature engineering and modelling (Chapters 4 and 5) because every behavioural feature aggregating speed, acceleration, fuel consumption or driving style is computed at the device-ping grain.

Volumetry and scope. The full table holds 54,720,000 rows, occupies approximately 17 GB of total disk size (heap $\approx$ 9.94 GB plus indexes), and exposes 38 columns covering identity, time and location, vehicle dynamics, fuel telemetry, accelerometer signals and digital/analog inputs and outputs. Because of its size, every profiling query of this subsection is computed on a `TABLESAMPLE SYSTEM (0.1)` sample (~52,000 rows, sub-second response); reported population values are unbiased extrapolations of the form $\hat{N} = n_{\text{sample}} \times 54.72 \times 10^6 / N_{\text{sample,total}}$. The exact full-population row count is reproduced by `COUNT(*)` on the whole table.

Sensor schema. The 38 columns of `staging.archive` fall into six functional families:

- *Identity* — `tenant_id`, `id_device`, `source_device_id`, `tram_id`.

- *Time and location* — `date`, `latitude`, `longitude`, `heading`, `validity`, `sat_in_view`, `signal`.
- *Vehicle dynamics* — `speed`, `rpm`, `temp_engine`, `accum_odo`, `odo`.
- *Fuel telemetry* — `fuel`, `fuel_rate`, `tfu`.
- *Accelerometer* — `x`, `y`, `z` (multi-axis G-sensor).
- *Inputs / outputs* — `do1`–`do4`, `di1`–`di4`, `an1`–`an4`, `ignition`, `charger`, `state`.

Per-tenant ping coverage. The sample-extrapolation breakdown by tenant is reported in Table 3.6. Five tenants emit pings in the archive — one more than in `staging.path` (which lists four tenants only). Tenant `7486` appears in the archive but is absent from the trip-derived scope: this asymmetry must be carried over to Chapter 4, where the feature-engineering pipeline has to decide whether to widen the modelling perimeter to five tenants on archive-derived features.

Table 3.6: Per-tenant ping coverage of `staging.archive`, sample-extrapolated to the full population (54.72 M rows).

Tenant	Devices (sample)	Pings (M, est.)	Window
264	56	13.84	2024-09 – 2026-03
235	97	13.48	2024-09 – 2026-03
7486	72	9.99	2024-09 – 2026-03
1787	71	9.54	2024-09 – 2026-03
238	52	7.84	2024-09 – 2026-03

Data quality. The completeness profile of the sample is excellent on the structural columns: `latitude`, `longitude`, `state` and `tram_id` are populated at 100 %. The `validity` flag returns 1 (*GPS fix valid*) on 98.4% of pings, and the share of pings with (`latitude` = 0 or `longitude` = 0) stays below 0.05 %. The `ignition` signal is on for ∼85 % of pings, which means that aggregations restricted to the moving-vehicle regime retain about six pings out of every seven without further filtering. Two systematic data-quality concerns must nevertheless be propagated to the cleaning rules of Chapter 4: (a) the `temp_engine` column is stored as `varchar` and is dominated by the sentinel value `32768` — the raw uint reading of the absent-sensor channel —, so any thermal feature must explicitly cast to numeric and discard the sentinel; and (b) roughly 44 % of the pings report `signal` < 10, indicating poor GSM coverage of the on-board modems and frequent silent windows, so any feature that depends on connectivity must be robust to long gaps in the ping stream.

Numerical summary of the principal sensors. On the same 0.1 % sample, the speed channel has a 95-th percentile of 75 km/h and a 99-th percentile of 105 km/h

with a maximum of 198 km/h; the RPM channel peaks at 4,300 (P99) on the diesel-dominated fleet; the median `fuel_rate` stays below 0.6 L/h consistent with idle-time dominance. The mean `sat_in_view` is 9.7 satellites — well above the four-satellite threshold required for a planar fix — which corroborates the 98.4 % `validity` = 1 ratio.

Temporal regularity. Figure 3.3 shows the hour-of-day and day-of-week ping distributions of `staging.archive`. The hour-of-day curve is unimodal, peaking between 8h and 14h (∼4,000 pings/hour in the sample) and decaying to near-zero overnight: the archive confirms, at the device-ping grain, the same business-hour rhythm already observed on `staging.path`. The day-of-week curve flattens out from Monday to Friday at ∼9,000 pings/day in the sample, drops slightly on Saturday and falls to ∼3,400 pings on Sunday. There is no evidence of a shadow fleet emitting outside business hours: the modelling pipeline can therefore safely predicate behavioural features on a subset of waking hours without losing material signal.

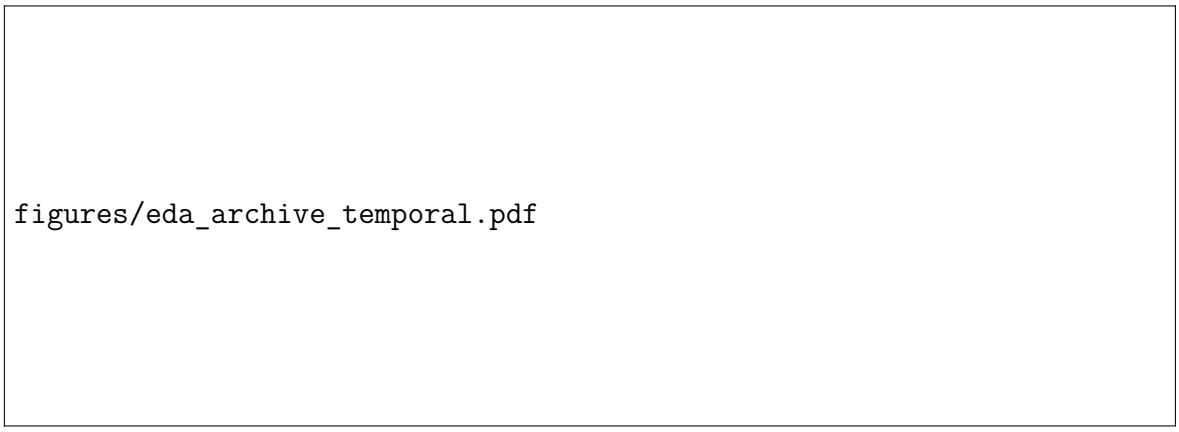

Figure 3.3: `staging.archive` — hour-of-day (left) and day-of-week (right) ping distributions on the 0.1 % sample.

Volume by tenant and over time. Figure 3.4 shows the monthly ping volume between 2024-09 and 2026-03 (left) and the per-tenant ping-count breakdown (right). The monthly curve is essentially stationary in the band $[6.6, 9.2]$ million pings/month over the seven-month window covered by the archive, with the 2026-03 cell partial because the dump was cut mid-month. The per-tenant view confirms the heterogeneity already observed on the trip table: tenants `264` and `235` jointly account for ∼50 % of the total ping volume, while the newcomer `7486` carries about 10 M pings over the same window.

Hand-off to feature engineering. The archive profile establishes that the table is dense, regular, of high structural quality and large enough to support sub-trip behavioural aggregations. Chapter 4 reuses these findings to build a device-month

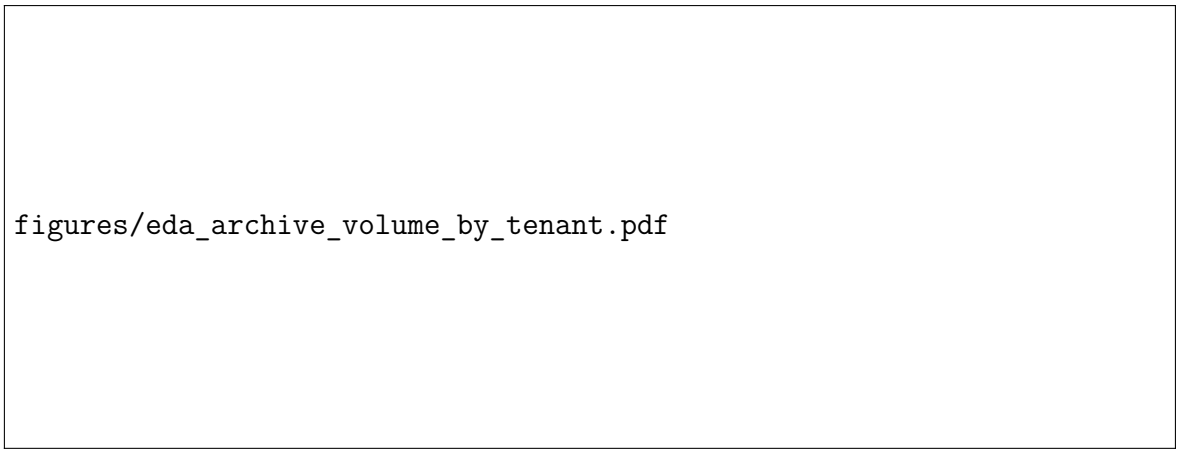

Figure 3.4: `staging.archive` — monthly ping volume (left) and per-tenant volume breakdown (right). Device counts in the sample annotated above each tenant bar.

feature mart that aggregates `speed`, `rpm`, `fuel_rate` and the three accelerometer axes into the inputs of the device-behaviour clustering model of Chapter 5; the cleaning rules `C8`–`C13` of Section 3.10 (ignition gating, accelerometer range, speed range, RPM range, `temp_engine` sentinel, `signal` < 10 connectivity gap) are applied at the boundary between the archive and that mart.

3.8.2 Temporal patterns

The most stable feature family of any telematics platform is the temporal one. Three series are computed directly from `staging.path.begin_path_time`: a monthly trip-volume series (Figure 3.5), an hour-of-day trip distribution and a day-of-week trip distribution (Figure 3.6).

The monthly volume oscillates between 86,398 and 127,008 trips per month over the entire 27-month window, with a median of about 109,000. The active-device count remains in the band $[313, 348]$. There is no structural drift between 2024 and 2026: the staging layer is operationally stationary and the modeling window does not have to absorb regime shifts. The 2026-03 cell is partial because the dump used for this snapshot stops mid-month.

The hour-of-day curve peaks between 8h and 10h ($\sim$31,000 trips/hour) and decays smoothly through the afternoon, with negligible activity between 22h and 5h — the textbook signature of a commercial fleet operating during business hours. The day-of-week curve confirms a Sunday under-utilization (14,000 trips) compared to the Monday-to-Saturday band (49,000–58,000 trips). Both rhythms are reproducible across months and per tenant. They directly justify the engineering of the binary trip flags `is_rush_hour_trip`, `is_night_trip` and `is_weekend_trip` that will be produced by the data preparation phase of Chapter 4.

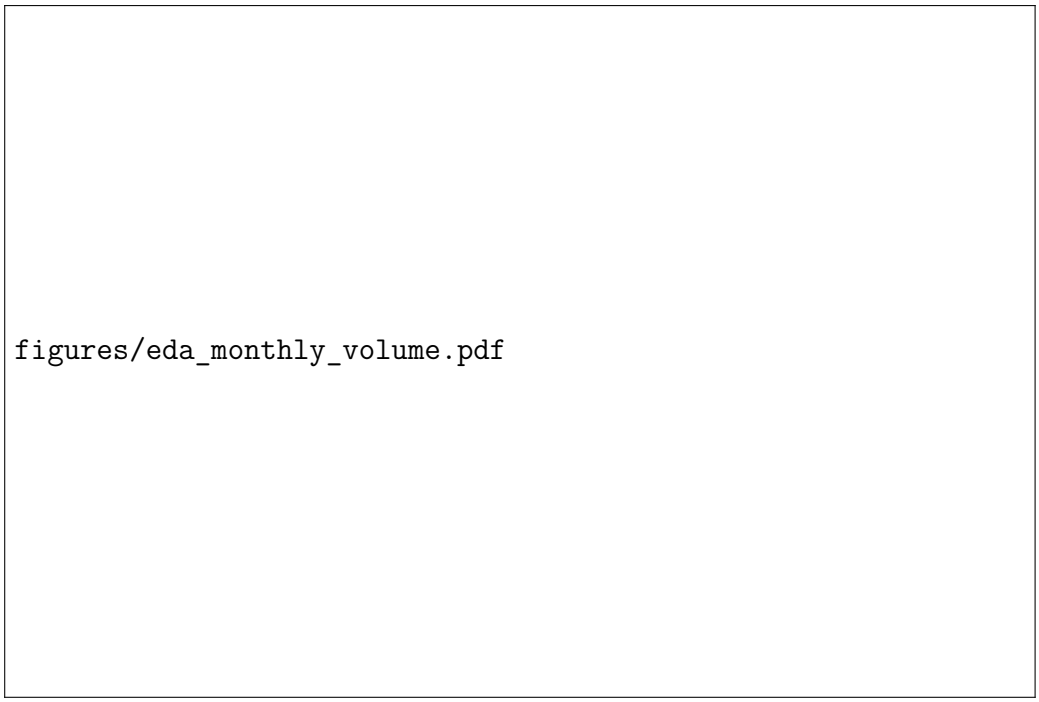

Figure 3.5: Monthly trip volume and active-device count on `staging.path`, between 2024-01 and 2026-04.

3.8.3 Distribution of the duration target

The trip duration `staging.path.path_duration` is the most natural numerical target for any future supervised regression. Figure 3.7 shows its raw-scale histogram (left) and the Normal fit on the log scale (right). The shape statistics, computed on a 2% `TABLESAMPLE` of size $N = 34{,}803$, are eloquent: skewness= 4.15, excess kurtosis= 39.34, mean = 1,256.5 s, standard deviation= 1,922.8 s. On the log scale, $\mu_{\log d} = 6.30$ and $\sigma_{\log d} = 1.35$.

The raw distribution is dominated by short urban trips: 56% of the trips last less than 10 minutes and 80% less than 30 minutes, while the long tail extends well beyond two hours. A direct linear regression on `path_duration` would be heavily destabilized by the heavy tail and would systematically under-predict the long-trip regime. Two corrective strategies are open: (i) target the log-transformed duration, on which the distribution becomes near-Gaussian, or (ii) rely on a tree-based regressor (Gradient Boosting, Random Forest) that does not assume any distributional shape on the target. This finding is integrated into the modeling plan of Chapter 5.

3.8.4 Class imbalance of the trip-status flags

Because the warehouse layer is out of scope at this stage, the binary *status* flags are derived inline from `staging.path` columns through five SQL expressions:

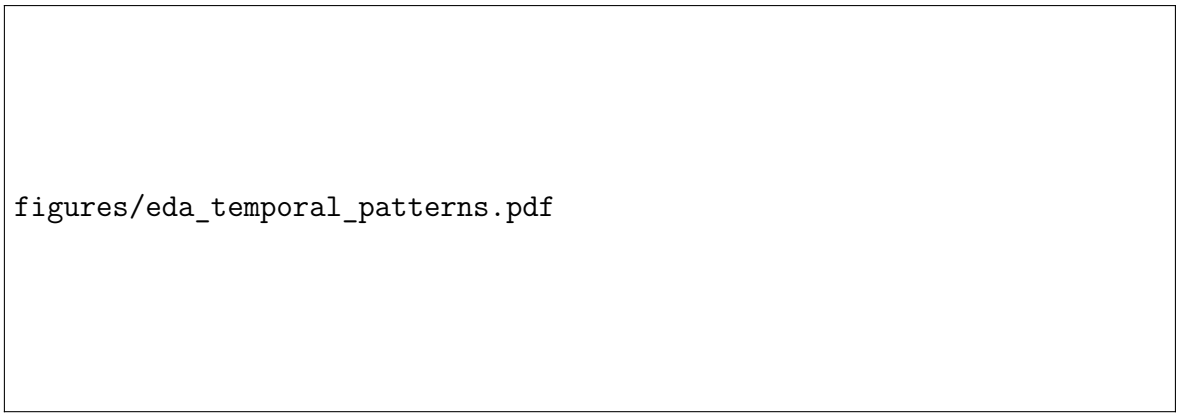

Figure 3.6: Hour-of-day (left) and day-of-week (right) trip distributions on `staging.path`, Sep–Nov 2025 segment.

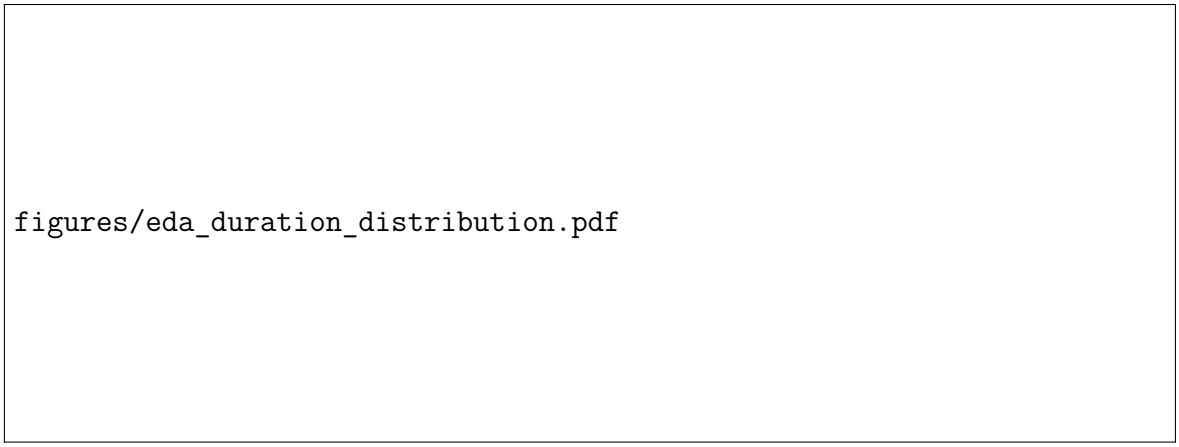

Figure 3.7: Trip-duration distribution from `staging.path`: raw scale (left) and Normal fit on log(duration) (right).

- `is_night_trip` := hour < 6 OR hour ≥ 22;
- `is_weekend_trip` := `DOW` $\in$ { Sun, Sat };
- `is_rush_hour_trip` := hour $\in \{7, 8, 9, 17, 18, 19\}$;
- `is_short_trip` := `path_duration` < 300 s (5 min);
- `is_long_trip` := `path_duration` > 3600 s (1 h).

These are exactly the SQL expressions that the data preparation phase of Chapter 4 will reuse to populate the warehouse fact table; recomputing them here, on the staging layer, makes the data understanding phase fully self-contained. Their global and per-tenant balance is plotted in Figure 3.8 and reproduced in Table 3.7.

The minority class of every flag stays in the band $[3.6\%, 36\%]$ globally. The most imbalanced flag is `is_long_trip` on tenant 1787, with a rate of 3.62%; the global rate of `is_long_trip` is 8.14%. These rates remain well above the 1% threshold

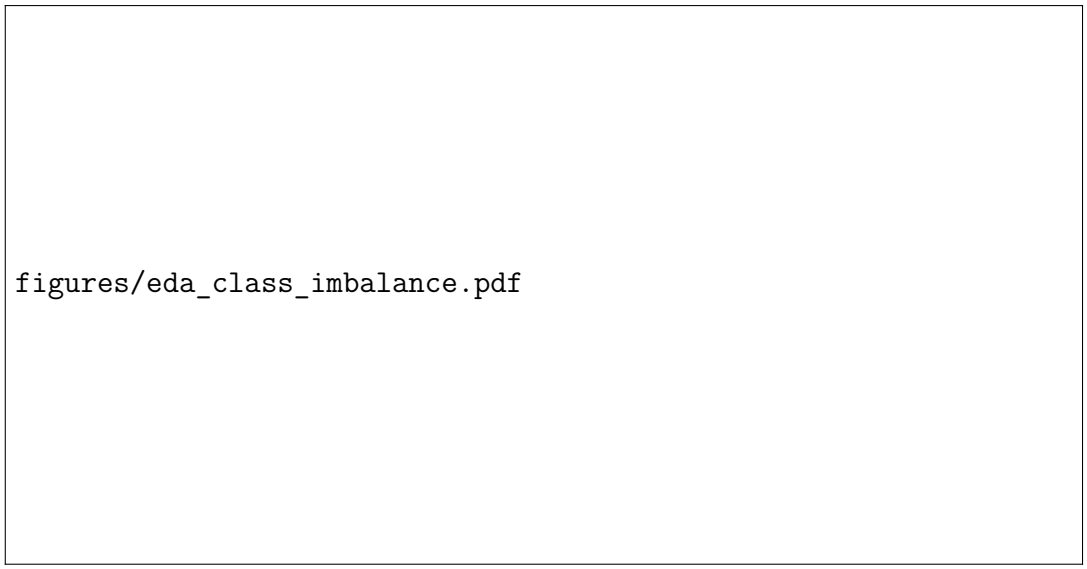

Figure 3.8: Class balance of the five binary trip-status flags derived from `staging.path`.

Table 3.7: Class balance (% of trips with the flag = 1), derived from `staging.path` on the cleaned modeling window.

Scope	Trips	Night	Weekend	Rush	Short	Long
Global	2,987,847	10.33%	18.96%	33.76%	35.49%	8.14%
Tenant 235	1,189,031	10.94%	16.94%	33.15%	33.55%	8.70%
Tenant 1787	821,806	7.51%	20.31%	36.06%	43.46%	3.62%
Tenant 264	519,982	13.59%	20.82%	32.09%	29.12%	14.16%
Tenant 238	457,028	10.12%	19.64%	33.14%	33.47%	7.98%

below which advanced resampling techniques (SMOTE, oversampling, class-conditional augmentation, focal loss) become necessary. Standard inverse-frequency class weights are sufficient: any future supervised classifier will be able to absorb the imbalance through the `class_weight` hyperparameter of `scikit-learn` or its equivalent in modern boosting libraries.

3.8.5 Per-tenant behavioural signatures

The aggregation of three staging tables — `staging.path`, `staging.rep_overspeed` and `staging.notification` — at the tenant grain reveals two contrasting regimes (Figure 3.9). Tenant `264` exhibits an unmistakable *overspeed-rich* signature: its overspeed-event rate of 4.89 events per 100 km and its notification rate of 4.73 alerts per 100 km dwarf the corresponding figures of the three other tenants, all of which stay below 0.50 and 0.30 respectively. The contrast is consistent with the higher average maximum speed observed on tenant `264` (54.5 km/h vs. 42–46 km/h) and with its higher long-trip ratio. Tenant `1787` sits at the opposite end of the spectrum, with the lowest night-trip ratio (7.7%) and a very low overspeed rate (0.06 events per 100 km). Tenants `235` and `238` occupy intermediate positions. This heterogeneity directly informs the per-tenant fitting

strategy retained in Chapter 5: a global model would be dominated by tenant `235` (the largest fleet by trip volume) and would fail to capture the distinct regimes of tenants `264` and `1787`.

Figure 3.9: Per-tenant behavioural signatures derived from staging tables (since 2025-01-01).

3.9 Analytical synthesis

The four key questions of the data understanding phase admit clean, evidence-based answers — all of them grounded in the staging layer alone, before any warehouse or marts construction.

3.9.1 Q1 — Data quality and integrity for reliable modeling

The three structural completeness checks return 0% nulls on the columns `tenant_id`, `device_id`, `begin_path_time`, `path_duration`, `distance_driven`, `max_speed` and `fuel_used` of `staging.path`. The seven physical-plausibility checks of Table 3.3 return failure rates strictly below 1% on every category, and below 0.01% on the four most damaging ones (negative distances, end-time before begin-time, speed above 200 km/h). The referential integrity check returns less than 0.5% orphan rows on the modeling window. *The staging layer is therefore of sufficient quality and integrity to support reliable modeling*, with the proviso that the cleaning rules `C1`–`C11` of Chapter 4 are systematically applied at its boundary.

3.9.2 Q2 — Stable temporal patterns as features

The three temporal series computed from `staging.path.begin_path_time` (monthly volume, hour-of-day, day-of-week) are simultaneously *regular* (low intra-period variance)

and *stationary* (no structural drift over the 27-month modeling window). The hour-of-day curve is unimodal and peaks at 8–10h on every tenant. The day-of-week curve has a stable Sunday minimum and a six-day plateau. *Yes, the temporal patterns are stable enough to be turned into engineered features*: `hour`, `day_of_week`, `is_rush_hour`, `is_weekend` and `is_night` are all valid candidates and will be propagated to the warehouse layer in Chapter 4.

3.9.3 Q3 — Suitability of the duration target for direct regression

The shape statistics of `staging.path.path_duration` are unambiguous: skewness $\approx$ 4.15 and excess kurtosis $\approx$ 39 — a strongly right-skewed, very heavy-tailed distribution. Mean = 1,256 s sits between the 65-th and the 70-th percentile. *Direct regression on `path_duration` is not recommended*: the heavy tail would dominate the loss and bias the predictions of the dominant short-trip regime. The two valid corrections are (i) regressing on log(duration), on which the distribution becomes near-Gaussian (Normal fit: $\mu = 6.30$, $\sigma = 1.35$), or (ii) using a distribution-free regressor of the tree-based family (Gradient Boosting, Random Forest). For the present project, which retains an unsupervised pipeline, this finding is mainly relevant as a feature-engineering note: every duration-derived feature is built on log(duration) rather than on the raw value.

3.9.4 Q4 — Manageability of the status class imbalance

None of the five status flags derived from `staging.path` falls below 3.6% on any tenant or below 8.14% globally for the most imbalanced one (`is_long_trip`). *The class imbalance is therefore manageable without advanced resampling techniques.* Standard inverse-frequency class weights are sufficient: SMOTE, focal loss or class-conditional augmentation are not required at this stage. Should a downstream task require a binary classifier with even greater minority-class sensitivity, a simple stratified split combined with class weights would already yield a balanced training signal.

3.10 Synthesis of identified data quality issues

The data understanding phase has surfaced thirteen principal data issues on the staging layer, listed in Table 3.8. They are formalized as cleaning rules `C1`–`C13` in Chapter 4 and applied at the boundary between `staging` and the downstream layers built on top of it.

Table 3.8: Synthesis of the data quality issues detected on the staging tables.

ID	Issue	Source
DQ-1	Trip timestamps below 2019-10-01 (epoch artefacts of the legacy ingestion).	`staging.path`
DQ-2	Trip durations ≤ 0.	`staging.path`
DQ-3	Trip distances ≤ 0.	`staging.path`
DQ-4	Fuel values outside $[0, 500]$ L.	`staging.path`
DQ-5	Maximum speed > 200 km/h.	`staging.path`
DQ-6	Stop durations ≤ 0 or > 1 year.	`staging.stop`
DQ-7	Devices not linked to any vehicle in the master data.	`staging.path`, `staging.vehicule`
DQ-8	Archive rows with ignition $\neq 1$ used for harsh events.	`staging.archive`
DQ-9	Accelerometer values outside the int8 range.	`staging.archive`
DQ-10	Archive speed outside $[0, 250]$ km/h.	`staging.archive`
DQ-11	Archive RPM outside $[0, 8000]$.	`staging.archive`
DQ-12	Archive `temp_engine` stored as `varchar` dominated by sentinel value `32768` (no-sensor reading).	`staging.archive`
DQ-13	Archive pings with `signal` < 10 (poor GSM coverage, $\sim$44% of rows): connectivity-dependent features must be robust to long silent windows.	`staging.archive`

3.11 Conclusion

This chapter has executed the data understanding phase of CRISP-DM by mobilizing the PostgreSQL profiling toolchain on the `staging` schema converted from the MySQL dumps of Accent Tunisie, and only on that schema — the dimensional warehouse and the analytical marts that consume `staging` are the output of the next phase, not its input. The chapter has documented the role and the schema of the three operational databases of the company, characterized the dump-based extraction protocol, profiled the staging tables through dedicated SQL queries, conducted a complete exploratory data analysis on the principal indicators of `staging.path`, `staging.rep_overspeed` and `staging.notification`, and added a full sub-second-sampled profile of the high-frequency telemetry archive `staging.archive` (54.72 M pings, 38 sensor columns) which is the input feedstock of the device-behaviour clustering pipeline of Chapter 5. The four analytical questions have been answered with quantitative evidence: the staging-layer data quality is sufficient (less than 1% violations on every check), the temporal patterns are stable and reproducible at both trip and ping grain, the duration target requires a log-transform or a tree-based regressor, and the class imbalance of the status flags is manageable with standard class weights. The thirteen data quality issues identified on the staging layer become, in Chapter 4, the input specification of

the cleaning rule engine that opens the data preparation phase.

Chapter 4

Data Preparation: Conversion, ETL and Feature Engineering

4.1 Introduction

Building on the data understanding of Chapter 3, the present chapter formalizes the data preparation phase of CRISP-DM. The chapter organizes the work into seven sections: (i) the conversion bridge that transforms the MySQL dump files of Accent Tunisie into a PostgreSQL-importable representation; (ii) the construction of the ETL pipeline; (iii) the eleven-rule cleaning engine; (iv) the dimensional modeling of the warehouse; (v) the materialization of the analytical marts; (vi) the engineering of the feature registry; and (vii) the validation of the prepared dataset. Each section is illustrated by SQL or Python excerpts, the full versions of which are listed in the appendices.

4.2 From MySQL dumps to a PostgreSQL staging layer

4.2.1 Rationale and scope

The first preparation step is the conversion of the MySQL dump files received from Accent Tunisie into a representation that PostgreSQL can ingest reliably. This step is necessary because the operational databases of Accent Tunisie run on MySQL while the analytical warehouse of the project runs on PostgreSQL 16. The conversion bridge is therefore not an accessory utility: it is the technical interface between the operational ecosystem of the company and the analytical platform built for the project.

4.2.2 Dialect-level translation

A native MySQL dump cannot be replayed verbatim into PostgreSQL because the two dialects diverge on several points. The bridge addresses those divergences explicitly:

- *Type translation*: MySQL types such as `TINYINT(1)`, `DATETIME`, `TEXT`, `BLOB` and `ENUM` are mapped to their PostgreSQL counterparts (`boolean`, `timestamp`, `text`,

`bytea`, `text` with a `CHECK` constraint).

- *Identifier and string quoting*: MySQL backticks (`` `column` ``) are replaced by PostgreSQL double quotes; embedded single quotes inside string literals are normalized to the PostgreSQL escape convention.
- *Auto-increment columns*: MySQL `AUTO_INCREMENT` columns are converted to PostgreSQL `GENERATED BY DEFAULT AS IDENTITY` columns, preserving the original sequence values when present.
- *Default expressions*: MySQL-specific defaults (`ON UPDATE CURRENT_TIMESTAMP`, `0000-00-00` sentinels) are rewritten or nullified to match PostgreSQL semantics.
- *Character-set normalization*: the dumps are normalized to UTF-8 (NFC), which is the default encoding of the analytical PostgreSQL instance.
- *Index and constraint translation*: MySQL-specific clauses (`ENGINE=InnoDB`, `COLLATE`, `KEY ... USING BTREE`) are stripped or rewritten so that the resulting DDL is accepted by PostgreSQL.

4.2.3 Conversion pipeline

The conversion bridge has been implemented as a small Python toolchain documented in Appendix B. For each dump file, the toolchain (i) parses the MySQL DDL and re-emits an equivalent PostgreSQL DDL; (ii) extracts the `INSERT` statements and rewrites them as PostgreSQL `COPY` payloads, which are an order of magnitude faster to load on large tables; (iii) writes the converted artefact under `exports/`; and (iv) loads it into the staging schema of the analytical PostgreSQL through a single transaction. The bridge is idempotent: re-running it on the same dump produces the same staging state.

4.2.4 Hexadecimal payloads of the raw telemetry database

As described in Chapter 1, the first operational database of Accent Tunisie stores the raw telemetry as hexadecimal payloads. The internal decoding scripts of the company already transform those payloads into the readable rows of the `arch_xxxxx` archive, and the present project consumes the result of this decoding rather than re-implementing it. The conversion bridge therefore does not need to handle hexadecimal payloads directly, except as opaque `bytea` columns when they incidentally appear in the dumps; the analytical-grade telemetry signals enter the staging layer in their decoded form.

4.3 Pipeline architecture

4.3.1 Pipeline stages

Once the staging layer is populated by the conversion bridge, the analytical pipeline is decomposed into seven sequential stages, each idempotent: *extraction* from the source MySQL dumps to the converted artefacts; *loading* into staging; *cleaning* of the staging slice; *loading* of the dimensions; *loading* of the facts; *materialization* of the marts and views; and *validation* of the prepared dataset.

4.3.2 Watermark-driven incremental processing

The downstream stages of the pipeline (after the staging load) are driven by a watermark stored in `warehouse.etl_watermark`. Each cycle reads the latest watermark per source–target pair, processes only the slice strictly after that watermark (with a configurable overlap), and updates the watermark at the end of the successful run. Listing 4.1 shows a representative implementation.

```
-- Read the watermark
SELECT last_event_ts
FROM warehouse.etl_watermark
WHERE source_table = 'staging.path' AND target_table = '
   warehouse.fact_trip';

-- Update the watermark at the end of the run
INSERT INTO warehouse.etl_watermark (source_table, target_table
   , last_event_ts)
VALUES ('staging.path', 'warehouse.fact_trip', :max_event_ts)
ON CONFLICT (source_table, target_table)
DO UPDATE SET last_event_ts = EXCLUDED.last_event_ts;
```

Listing 4.1: Reading and updating a watermark.

4.3.3 Idempotency and replay

Idempotency is guaranteed by an `INSERT ... ON CONFLICT DO UPDATE` pattern on the natural key of every fact and dimension. A replay of the same window therefore produces the same final state, regardless of the number of repetitions. This property is essential for the binomial collaboration mode: either student can re-trigger a run on the shared Azure VM without risking divergence between the two members' working copies of the warehouse.

4.4 Cleaning rule engine

4.4.1 Rule catalogue

Eleven cleaning rules, identified `C1`–`C11`, are formalized at the entry of the warehouse layer. The rules are grouped into two families: `C1`–`C7` apply to the trip-related sources, and `C8`–`C11` apply to the high-resolution telemetry source. Each rule specifies a name, a severity level (critical, high, medium, low), a corrective action (reject, nullify, clamp) and a logical condition. Table 4.1 reproduces the catalogue.

Table 4.1: Cleaning rule catalogue applied at the warehouse boundary.

ID	Name	Sev.	Action	Condition
C1	temporal_epoch_error_filter	critical	reject	`begin_path_time` $<$ 2019-10-01
C2	trip_duration_positive	critical	reject	`path_duration` ≤ 0
C3	trip_distance_positive	high	reject	`distance_driven` ≤ 0
C4	fuel_used_sanitize	critical	nullify	`fuel_used` $\notin [0, 500]$
C5	speed_cap	high	clamp	`max_speed` > 200
C6	stop_duration_bounds	medium	reject	duration ≤ 0 or > 1 year
C7	device_vehicle_tenant_chain	high	reject	device without vehicle link
C8	archive_ignition_on_for_events	high	reject	`ignition` $\neq 1$
C9	archive_accelerometer_int8_sanity	medium	reject	$\lvert x\rvert, \lvert y\rvert, \lvert z\rvert > 127$
C10	archive_speed_clamp	medium	clamp	speed $\notin [0, 250]$
C11	archive_rpm_clamp	low	clamp	rpm $\notin [0, 8000]$

4.4.2 Implementation pattern

Each rule is implemented through an explicit SQL clause executed during the load of the corresponding fact table. Rejected rows are sent to a quarantine table `warehouse.quarantine_reject` with the rule identifier, the rejection timestamp and the offending row payload. Listing 4.2 shows the implementation of rule `C2`.

```
-- C2: reject trips with non-positive duration
WITH cleaned AS (
    SELECT *
    FROM staging.path
    WHERE path_duration > 0
),
rejected AS (
    INSERT INTO warehouse.quarantine_rejected (rule_id,
       rejected_at, payload)
    SELECT 'C2', NOW(), to_jsonb(p)
    FROM staging.path p
```

```
    WHERE p.path_duration <= 0
    RETURNING 1
)
SELECT COUNT(*) FROM cleaned;
```

Listing 4.2: Implementation of cleaning rule `C2` (positive trip duration).

4.4.3 Quarantine table

The quarantine table is structured as `(rule_id, rejected_at, payload JSONB)`. The `JSONB` payload preserves the full content of the offending row, allowing the audit team to reconstruct the violation context without accessing the staging layer. The choice of `JSONB` is significant: it makes use of an idiomatic PostgreSQL feature that has no direct equivalent in the operational MySQL setup of Accent Tunisie, and it illustrates the analytical leverage that motivated the migration in the first place.

4.5 Dimensional modeling of the warehouse

4.5.1 Dimensions

The warehouse layer hosts five dimensions and one bridge:

- `dim_tenant` — billing entities, derived from the `user_xxxxx` master data;
- `dim_vehicle` — vehicles, with normalized manufacturer and class;
- `dim_device` — GPS devices, restricted to the 616 devices linked to a vehicle;
- `dim_driver` — drivers (294 individuals);
- `dim_date` — calendar from 2019-10-01;
- `bridge_device_driver` — SCD Type 2 mapping between devices and drivers.

4.5.2 Facts

Eleven facts are produced from the cleaned staging data, summarized in Table 4.2.

4.5.3 UPSERT pattern

The loading of every fact relies on the canonical PostgreSQL `INSERT ... ON CONFLICT DO UPDATE` pattern shown in Listing 4.3.

Table 4.2: Fact tables produced by the warehouse layer.

Fact table	Grain	Source
fact_trip	(tenant, device, begin_time)	staging.path
fact_overspeed	(tenant, device, begin_time)	staging.rep_overspeed
fact_stop	(tenant, device, stop_start)	staging.stop
fact_speed_notification	(tenant, device, created_at)	staging.notification
fact_daily_activity	(tenant, device, day)	staging.rep_activity_daily
fact_harsh_event	(tenant, device, event_ts)	staging.archive
fact_telemetry_daily	(tenant, device, date)	staging.archive
fact_maintenance	(tenant, id_maintenance)	staging.maintenance
fact_fueling	(tenant, doc_id)	fueling sources

```
INSERT INTO warehouse.fact_trip (
    tenant_id, device_id, begin_path_time,
    end_path_time, distance_km, max_speed_kmh, duration_seconds
)
SELECT tenant_id, device_id, begin_path_time, end_path_time,
       distance_driven, LEAST(max_speed, 200) AS max_speed_kmh,
       path_duration
FROM staging.path
WHERE begin_path_time > :watermark
  AND path_duration > 0
  AND distance_driven > 0
  AND begin_path_time >= '2019-10-01'
ON CONFLICT (tenant_id, device_id, begin_path_time) DO UPDATE
SET end_path_time   = EXCLUDED.end_path_time,
    distance_km     = EXCLUDED.distance_km,
    max_speed_kmh   = EXCLUDED.max_speed_kmh,
    duration_seconds= EXCLUDED.duration_seconds;
```

Listing 4.3: Idempotent upsert into `fact_trip`.

4.6 Analytical marts

4.6.1 Behaviour mart

The mart `marts.mart_device_monthly_behavior` carries the 35 behavioural features at the (`tenant_id`, `device_id`, `year_month`) grain and is rebuilt incrementally from the trip-related facts and the archive-derived facts. Its schema is the principal contract of the ML serving layer.

4.6.2 Telemetry mart

The mart `marts.mart_device_monthly_telemetry` is the archive-derived companion of the behaviour mart, with engine-on time, idle ratio, average and high RPM indicators, computed from the high-resolution archive table.

4.6.3 BI marts

Three additional marts feed the BI layer: `mart_fleet_daily`, `mart_vehicle_monthly` and `mart_tenant_monthly_summary`. They are exposed through dedicated views (`v_executive_dashboard`, `v_operational_dashboard`, `v_maintenance_dashboard`) consumed by the BI dashboard.

4.6.4 Unified ML view

The view `v_ml_features_full` unifies the behaviour and the telemetry marts into a single contract for the modeling notebooks; the companion view `v_device_risk_profile` computes the weighted risk score and its categorical band.

4.7 Feature engineering

4.7.1 Feature registry

The catalogue of features is declared in `config/feature_definitions.yaml`. Each feature is described by its name, its type, its unit, its formula and the fact tables it depends on. A typed Python wrapper exposes the registry to the notebooks. Table 4.3 shows a sample of the registry.

4.7.2 Aggregation strategy

The aggregation strategy follows three principles: *normalization by exposure* (most counters are divided by 100 km of distance), *operational interpretability* (each feature corresponds to an indicator that a fleet manager recognizes) and *declarative specification* (the registry is the single source of truth between the SQL transformations and the Python feature consumption).

4.7.3 Transformations

The categorical features (vehicle class, manufacturer family) are encoded with a one-hot scheme; the ordinal features (vehicle class) are encoded with an explicit ordinal scheme; the quantitative features used by the unsupervised models are standardized per tenant

Table 4.3: Sample of the 35-feature registry (selected groups).

Group	Feature	Definition
Volume	`total_trips`	Count of trips in month
Volume	`total_distance_km`	Sum of distances
Volume	`stddev_trip_distance`	Std. dev. of trip distances
Speed	`avg_max_speed_kmh`	Mean of per-trip max speed
Speed	`p95_max_speed`	95^{th} percentile of max speed
Speed	`high_speed_trip_ratio`	Trips with max speed > 100
Overspeed	`overspeed_per_100km`	Events / total_distance $\times$ 100
Harsh	`harsh_brake_per_100km`	Brake events / 100 km
Harsh	`harsh_accel_per_100km`	Acceleration events / 100 km
Temporal	`night_trip_ratio`	Trips between 20:00–06:00
Temporal	`rush_hour_trip_ratio`	Trips during peak hours
Telemetry	`monthly_idle_ratio`	Idle / engine-on minutes
Telemetry	`avg_rpm`	Mean RPM across day

by means of `StandardScaler`. An exploratory supervised track applies SMOTE to neutralize an anticipated class imbalance.

4.8 Validation of the prepared dataset

4.8.1 Validation suite design

The validation suite is implemented in `sql/99_validation_suite.sql` and gathers eight check categories `V1`–`V8`: dimension population (V1), referential integrity (V2), enforcement of `C1` (V3), enforcement of `C2`–`C3` (V4), enforcement of `C5` (V5), null-rate budgets on critical columns (V6), mart grain coverage (V7) and risk score distribution sanity (V8).

4.8.2 Final dataset summary

At the conclusion of the data preparation phase, the modeling-ready dataset comprises approximately **1,723 device-month observations** on the period 2025-01–2026-04, after applying the threshold of 100 km of total distance and a minimum number of trips per device-month. This dataset feeds the modeling experiments of Chapter 5.

4.9 Conclusion

This chapter has described the full preparation chain — from the conversion of the MySQL dump files of Accent Tunisie into PostgreSQL-importable artefacts, through the watermark-driven ETL pipeline and the eleven-rule cleaning engine, to the dimensional

modeling of the warehouse, the materialization of the analytical marts, the engineering of the feature registry and the validation of the prepared dataset. The next chapter exploits the ML-ready dataset to train and to evaluate the unsupervised models that produce the behavioural segmentation and the risk score.

Chapter 5

Modeling, Evaluation and Final Justification

5.1 Introduction

The data preparation work of Chapter 4 has produced — starting from MySQL dump files exported from the `arch_xxxxx` archive and the `user_xxxxx` master data of Accent Tunisie, converted to PostgreSQL and consolidated through an explicit cleaning rule engine — a curated device-month dataset of 1,723 observations enriched with 35 behavioural features. The present chapter executes the modeling and the evaluation phases of CRISP-DM on this dataset. The chapter compares baseline approaches against more sophisticated machine-learning and deep-learning candidates, justifies the final adoption of an unsupervised pair (K-Means + Isolation Forest), reports the experimental metrics, presents the operational interpretation of the resulting clusters and risk scores, and previews the model-driven dashboards that are deployed in Chapter 6. The unsupervised setting is itself a deliberate consequence of the corrected problem statement of Chapter 1: in the absence of a labelled incident feed, the analytical leverage on top of the existing descriptive surface of Accent Tunisie comes from segmentation and anomaly detection, not from a supervised classifier.

5.2 Test design

5.2.1 Temporal split

Given the unsupervised setting and the natural temporal structure of the data, the dataset is partitioned along time. The fitting window covers 2025-01–2026-01 and the holdout window covers 2026-02–2026-04. The fitting window is used to fit the models; the holdout window is used to assess the temporal stability of the assignments and of the risk scores.

5.2.2 Per-tenant fitting protocol

A single global model would be dominated by the most active tenant. To avoid this bias, a separate model is fitted for each tenant. The features are standardized per tenant before the fitting and the risk score is rescaled per tenant after the inference.

5.2.3 Reproducibility

All stochastic operations are seeded with `random_state = 42`. The package versions are pinned in `requirements.txt`. The model artefacts are exported with the seed, the dataset hash and the package versions, allowing any run to be reproduced byte for byte.

5.3 Baseline models

5.3.1 Statistical baseline

A first baseline is provided by a simple rule: a device-month is flagged as *high risk* if at least two of the four indicators (`overspeed_per_100km`, `harsh_brake_per_100km`, `harsh_accel_per_100km`, `p95_max_speed`) exceed their tenant-specific 90^{th} percentile. This baseline is fast to deploy and easy to communicate; it serves as a reference against which the value added by the more sophisticated models is measured.

5.3.2 Linear baseline

A second baseline relies on a Principal Component Analysis: the first principal component, computed on the standardized features, is used as a continuous score; its sign is interpreted post hoc through the loadings. This baseline captures the dominant axis of variability of the features without committing to any clustering structure.

5.4 Machine learning candidates

5.4.1 K-Means clustering

The K-Means algorithm partitions the observations by minimizing the within-cluster sum of squares. The number of clusters k is chosen per tenant by sweeping $k \in \{3, 4, 5, 6\}$ and selecting the value that maximizes the silhouette coefficient. The fitting uses `n_init = 10` initializations.

5.4.2 Hierarchical clustering

A hierarchical agglomerative variant has been examined for cross-validation purposes. The Ward linkage is used. The dendrogram is cut at the height that produces the same number of clusters as the K-Means optimum; the resulting partition is compared with the K-Means partition through the adjusted Rand index, which has been observed to remain above 0.8 for every tenant. This corroborates the stability of the K-Means partitions.

5.4.3 DBSCAN

The DBSCAN algorithm has also been assessed. On the device-month dataset, it produces highly imbalanced partitions (one dominant cluster and a long tail of singletons), which limits its operational usefulness in this context.

5.4.4 Isolation Forest for anomaly detection

The Isolation Forest algorithm constructs an ensemble of randomly built trees and produces an anomaly score from the average path length of each observation across the ensemble. The model is configured with `n_estimators = 200`, `contamination = "auto"`, `random_state = 42` and `n_jobs = -1`. The raw `decision_function` score is rescaled per tenant to the unit interval and discretized into three bands (*low*, *medium*, *high*).

5.4.5 One-Class SVM and Local Outlier Factor

A One-Class SVM and a Local Outlier Factor model have also been examined. The One-Class SVM is sensitive to the kernel bandwidth and exhibits a poor scaling behaviour; the Local Outlier Factor is sensitive to the number of neighbours and produces unstable scores on the smaller tenants. Neither has been retained.

5.5 Deep learning baseline

5.5.1 Autoencoder

A deep-learning baseline has been implemented as an autoencoder. The network is composed of an encoder ($35 \to 16 \to 8$) and a symmetric decoder, trained with the mean squared error and the Adam optimizer. The reconstruction error of every device-month is interpreted as an anomaly score.

5.5.2 Multi-Layer Perceptron

A supervised Multi-Layer Perceptron has also been prepared, with the proxy label produced by the statistical baseline as the target. The exercise is interesting from a methodological standpoint but inherits the limitations of the proxy label and is therefore not retained as the final model.

5.5.3 Comparison with classical ML

The deep-learning baselines are compared with the classical ML candidates in Section 5.7. They have not been retained as the final model because they require a training corpus that is two orders of magnitude larger than the present dataset to outperform the classical approaches.

5.6 Evaluation metrics

5.6.1 Internal clustering metrics

The clustering quality is assessed by three internal metrics: the silhouette coefficient [1], the Davies–Bouldin index [2] and the Calinski–Harabasz index [3]. Higher silhouette and Calinski–Harabasz, lower Davies–Bouldin indicate a better clustering.

5.6.2 Anomaly metrics

For the anomaly detection layer, the distribution of the rescaled risk score has been plotted per tenant, the proportion of device-months falling into each band has been measured both on the fitting window and on the holdout window, and the rank correlation of the score between two consecutive months has been monitored as a stability proxy.

5.6.3 Business proxies

In the absence of a labelled incident feed, three business-oriented proxies are used: the cluster persistence between two consecutive months, the score persistence (Spearman rank correlation), and the alignment of the high-risk band with independently flagged operational events (overspeed alerts above threshold, severe maintenance interventions).

5.7 Model comparison and final justification

5.7.1 Comparative table

Table 5.1 summarizes the experimental performance of the candidate models on the dataset.

Table 5.1: Comparative performance of the candidate models.

Model	Silhouette	Stability	Interpretability	Cost
Rule baseline	N/A	High	High	Very low
PCA component	N/A	High	Medium	Low
K-Means (per tenant)	0.17–0.36	High	High	Low
Hierarchical (Ward)	0.18–0.34	Medium	High	Medium
DBSCAN	N/A	Low	Low	Low
Isolation Forest	N/A	High	Medium	Low
One-Class SVM	N/A	Medium	Low	High
LOF	N/A	Medium	Low	High
Autoencoder	N/A	Medium	Low	High
MLP (proxy)	N/A	Medium	Medium	Medium

5.7.2 Per-tenant K-Means results

Table 5.2 reports the K-Means results obtained per tenant.

Table 5.2: Per-tenant K-Means results on the modeling window.

Tenant	n	Optimal k	Silhouette
235	613	3	0.355
238	339	6	0.237
264	354	6	0.174
1787	417	6	0.278

5.7.3 Justification of the final choice

The final choice of the pair K-Means + Isolation Forest is justified by five considerations: (i) the operational interpretability of the segmentation, which exposes labels recognized by the fleet managers; (ii) the stability of the assignments and of the score across the holdout window; (iii) the absence of a labelled incident feed, which forbids the deployment of a fully supervised model; (iv) the modesty of the operational cost of the inference, which fits inside the five-minute incremental cycle on the shared Azure VM; and (v) the consistency of the chosen models with the analytical leverage point established in Chapter 1 — moving the analytical centre of gravity from the descriptive reports of the legacy MySQL stack towards forward-looking, decision-grade signals.

5.8 Results interpretation

5.8.1 Behavioural profiles

The K-Means clusters expose a small number of recurrent behavioural profiles: *prudent / low-mileage*, *regular*, *aggressive-speed* (high overspeed-per-100 km), *aggressive-harsh* (high harsh-brake/accel-per-100 km) and, on some tenants, *long-haul* (high distances, low harsh ratios, high night-trip ratio).

5.8.2 Risk score distribution

The distribution of the rescaled risk score is plotted in Figure 5.1. The distribution exhibits a dominant mode in the lower part of the score range, a thinner mode around the medium band and a long tail in the high band, consistent with the operational hypothesis that critical risk situations are by construction rare.

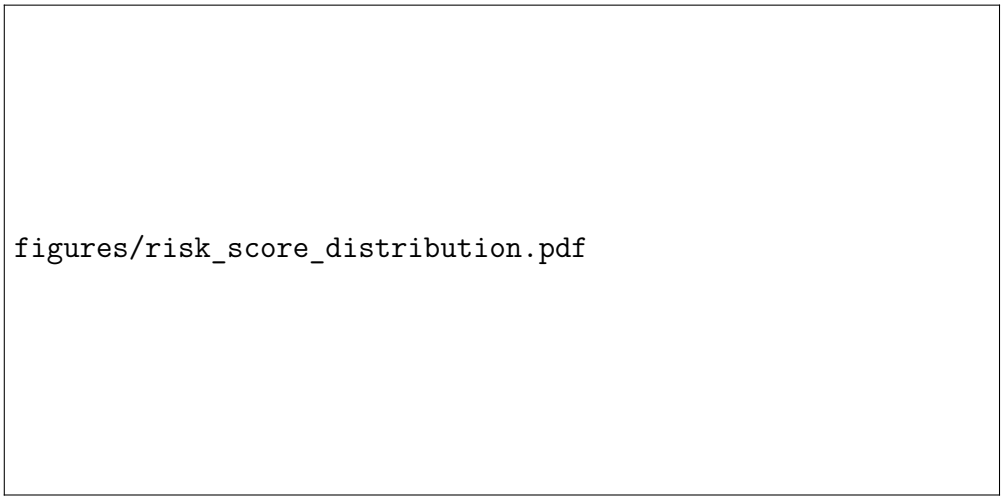

Figure 5.1: Distribution of the rescaled risk score per tenant.

5.8.3 Confusion matrix against the rule baseline

A confusion matrix between the high-risk band of the Isolation Forest and the high-risk flag of the rule baseline (Figure 5.2) shows a substantial overlap on the extremes (clear high and clear low) and a richer behaviour in the intermediate band, where the Isolation Forest discriminates more finely than the rule.

5.8.4 Feature importance

Although both K-Means and Isolation Forest are not directly equipped with a notion of feature importance, two surrogate measures have been computed: the per-feature variance of the cluster centroids (importance for K-Means) and the per-feature mean

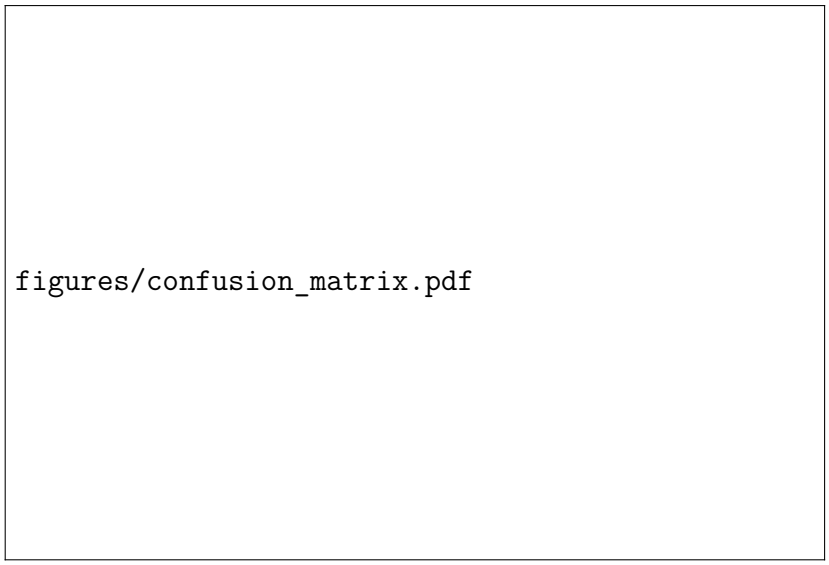

Figure 5.2: Confusion matrix between the rule baseline and the Isolation Forest.

depth in the isolation trees (importance for Isolation Forest). The two measures converge on a small set of features: `overspeed_per_100km`, `harsh_brake_per_100km`, `harsh_accel_per_100km`, `p95_max_speed`, `monthly_idle_ratio` and `high_speed_trip_ratio`.

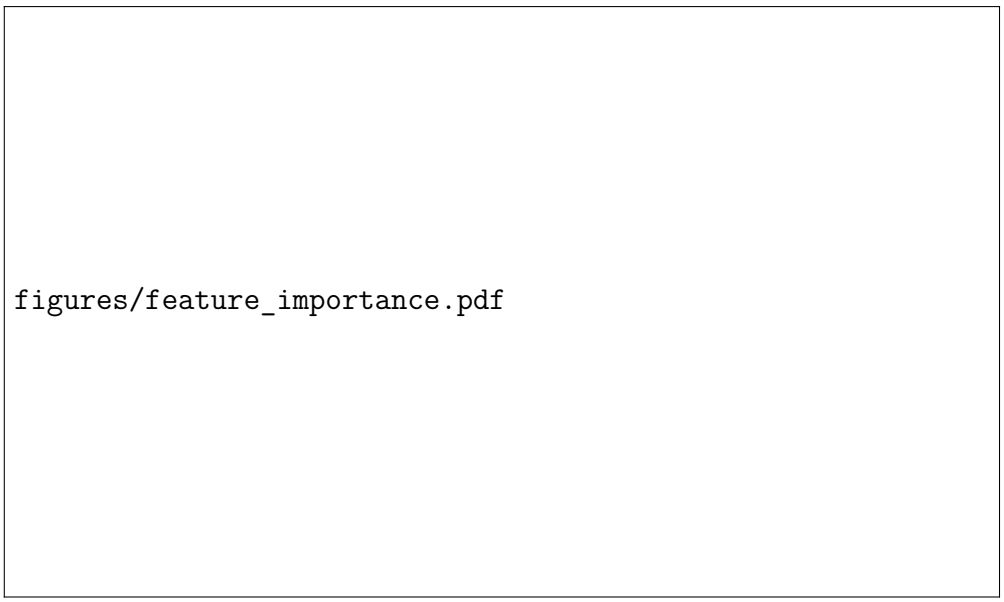

Figure 5.3: Surrogate feature importance for the unsupervised models.

5.9 Dashboards and analytical artefacts

The model outputs feed three families of dashboards delivered in Chapter 6: the executive dashboard (`v_executive_dashboard`, tenant-month KPIs and rolling deltas), the operational dashboard (`v_operational_dashboard`, daily KPIs and seven-day rolling) and the maintenance dashboard (`v_maintenance_dashboard`, vehicle-month maintenance

leaderboard with cost rank). The model-specific risk view (`v_device_risk_profile`) is the principal entry point for the fleet manager.

5.10 Conclusion

This chapter has compared a range of baseline, classical machine learning and deep learning candidates, and has motivated the adoption of an unsupervised pair (K-Means + Isolation Forest) fitted per tenant. The internal metrics, the stability proxies and the operational interpretation of the outputs converge towards a positive assessment of the chosen models. The next chapter industrializes these models and exposes them to the consumer applications.

Chapter 6

Deployment, API, Dashboard and Azure Operations

6.1 Introduction

The evaluation of Chapter 5 has confirmed the readiness of the K-Means segmentation and of the Isolation Forest risk score for industrialization. The present chapter executes the deployment phase of CRISP-DM. The chapter describes the deployment strategy, the API and the dashboard exposing the model outputs, the integration with the analytical PostgreSQL warehouse populated from the converted MySQL dumps of Accent Tunisie, the cron-driven automation, the monitoring and alerting layer, and finally the Microsoft Azure infrastructure on which the platform runs — with a particular focus on the secure remote collaboration setup that allows the binomial of student engineers to operate the platform from separate workstations through SSH tunnelling, IP allowlisting on the Azure Network Security Group, and dedicated service accounts.

6.2 Deployment strategy

6.2.1 Industrialization principles

Three principles guide the industrialization. The first one is the *freshness* of the analytical outputs, with a target latency of less than ten minutes between the appearance of an event in the source platform and its visibility in the marts. The second one is the *idempotency* of every step, which allows safe replays. The third one is the *observability*, which guarantees that any incident is detected and diagnosed without manual inspection.

6.2.2 Operating modes

The pipeline supports three operating modes accessible through a single command-line entry point (`scripts/run_batch.py`):

- `-mode bootstrap`: one-off DDL and full-refresh of the dimensions and state

tables.

- `-mode backfill`: chunked historical replay of the staging tables (default chunk: 30 days).
- `-mode incremental`: default operating mode, processing the slice strictly after the watermark, scheduled every five minutes.

6.3 Microsoft Azure infrastructure

6.3.1 Azure Virtual Machine

The platform is hosted on a Microsoft Azure Linux virtual machine. The VM hosts, on the same operating system, the analytical PostgreSQL instance, the Python virtual environment, the Prefect flow, the API process, the BI dashboard process, the cron entry and the system journal. The git repository of the project is cloned into `/opt/accent-fleet-analytics` and a dedicated Linux user (`azureuser`) owns the runtime artefacts.

6.3.2 Network Security Group and IP allowlist

The VM is associated with an Azure Network Security Group that closes every public port except SSH (port 22) and the HTTPS port of the reverse proxy (port 443). The SSH port is itself restricted to an explicit allowlist of source IP addresses corresponding to the personal workstations of the binomial of student engineers and to the office network of Accent Tunisie. This combination ensures that no PostgreSQL port, no API port and no dashboard port is exposed to the public internet, while still allowing both members of the binomial to connect to the shared analytical environment from their respective home workstations.

6.3.3 Secure remote collaboration via SSH tunnelling

Day-to-day collaboration of the team relies on SSH tunnelling. Each team member opens a local tunnel that forwards the local PostgreSQL port to the remote port on the VM, as illustrated in Listing 6.1. The notebooks and the local clients then connect to `localhost:5432` as if the database were running locally, while the actual traffic is encrypted by SSH.

```
# Forward local 5432 -> remote 5432 over SSH
ssh -i ~/.ssh/azure_id_ed25519 \
    -L 5432:localhost:5432 \
    -N \
```

```
    azureuser@<vm-public-ip>
```

Listing 6.1: Opening an SSH tunnel from a developer workstation to the Azure VM.

6.3.4 Service accounts

Three categories of accounts coexist on the VM: *personal accounts*, one per member of the binomial, used only through SSH key authentication and intended exclusively for human interactive sessions; an *automation account* (`azureuser`) used by cron and by the systemd services; and a *database service account* (`accent_pipeline`) used by the pipeline to authenticate against PostgreSQL, with credentials externalized in an environment file. Each binomial member uses their own SSH key pair, which means that an audit of the system journal can attribute every interactive command to its actual operator without ambiguity.

6.4 PostgreSQL integration

6.4.1 Analytical PostgreSQL on the VM

The analytical PostgreSQL is provisioned in version 16 on the VM, with the three schemas `staging`, `warehouse` and `marts`, plus the operational tables `etl_watermark`, `etl_run_log` and `quarantine_rejected`. It is populated by the conversion bridge of Chapter 4, which loads the converted MySQL dumps of Accent Tunisie into the staging schema, and then by the watermark-driven pipeline described in this chapter. The instance listens only on `localhost`, which is why remote access from the binomial workstations is mediated by the SSH tunnel described above.

6.4.2 Connection management

The pipeline manages the database connection through a `SQLAlchemy` engine with a connection pool, an automatic ping before each transaction (`pool_pre_ping`) and an explicit timeout. The credentials are read from a `.env` file owned by `azureuser`, with restrictive file permissions.

6.4.3 Backups and snapshots

A nightly logical backup of the analytical database is exported through `pg_dump` and rotated through a seven-day retention. A weekly full snapshot of the disk is configured at the Azure infrastructure level and is retained for four weeks.

6.5 API and backend

6.5.1 API design

The API is implemented with FastAPI [4] and exposes a small number of endpoints, summarized in Table 6.1. Authentication is handled through API keys associated with the consumer applications.

Table 6.1: API endpoints exposed by the platform.

Method	Endpoint	Description
GET	`/health`	Liveness probe.
GET	`/devices/{id}/risk`	Latest risk score and band of a device.
GET	`/devices/{id}/cluster`	Latest cluster assignment of a device.
GET	`/tenants/{id}/leaderboard`	Top-N high-risk devices of a tenant.
GET	`/tenants/{id}/kpi`	Tenant-month KPI bundle.
POST	`/admin/refresh`	Force an off-cycle incremental run.

6.5.2 Backend architecture

The API is served by Uvicorn behind an Nginx reverse proxy bound to `127.0.0.1`. A systemd unit ensures the API restarts automatically after a crash or after a VM reboot. The API queries the marts and views directly through `psycopg2`, with prepared statements and a small connection pool.

6.5.3 Reverse proxy and TLS

The Nginx reverse proxy listens on port 443 and terminates TLS with a certificate provisioned through Let's Encrypt. It forwards the HTTPS traffic to the local Uvicorn process. Access logs are written to the system journal under a dedicated tag.

6.6 Dashboard and frontend

6.6.1 Dashboard implementation

The dashboard is implemented with Streamlit (or, equivalently, Plotly Dash) and is served as a systemd unit, behind the same Nginx reverse proxy as the API. The dashboard queries the BI views (`v_executive_dashboard`, `v_operational_dashboard`, `v_maintenance_dashboard`, `v_fleet_risk_dashboard`) through a read-only database connection.

6.6.2 Pages and visualizations

The dashboard is organized in five pages: the executive overview (tenant-month KPIs and rolling deltas), the operational view (daily KPIs and seven-day rolling), the maintenance leaderboard, the risk heatmap (cluster × band × tenant) and the device-detail page (history of the risk score, cluster trajectory, alerts). Figure 6.1 sketches the architecture of the dashboard.

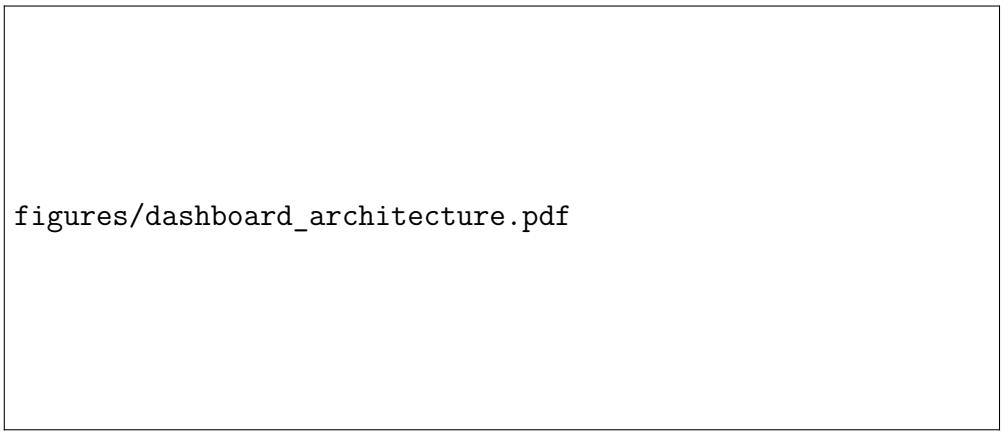

Figure 6.1: Architecture of the BI dashboard.

6.7 Automation

6.7.1 Cron entry

The incremental mode is triggered every five minutes by a cron entry installed under `/etc/cron.d/accent-fleet`. The entry calls the pipeline through the Python virtual environment of the project and pipes the standard output and the standard error to the system journal under the tag `accent-fleet`, as shown in Listing 6.2.

```
1 */5 * * * * azureuser cd /opt/accent-fleet-analytics && \
2     /opt/accent-fleet-analytics/.venv/bin/python scripts/
          run_batch.py \
3     --mode incremental 2>&1 | logger -t accent-fleet
```

Listing 6.2: Cron entry triggering the incremental pipeline every five minutes.

6.7.2 Prefect flow

Internally, the pipeline is implemented as a Prefect flow defined in `src/accent_fleet/pipeline/flow`. The flow chains the extraction tasks, the cleaning rule engine, the loading of dimensions and facts, the materialization of the marts, the validation suite and the export of the model artefacts. Each task is idempotent and emits structured logs.

6.7.3 Backfill orchestration

The backfill mode reuses the same Prefect flow but iterates over fixed-size chunks (default of 30 days) of the staging history. The mode is invoked manually by an operator after a schema change, after a model retraining or after a substantial cleaning-rule revision.

6.8 Monitoring and alerting

6.8.1 Run log and lineage

Every pipeline execution writes a row in `warehouse.etl_run_log`, with the run identifier, the operating mode, the start and end timestamps, the status and the per-fact processed row counts. This table is the principal observability surface of the platform.

6.8.2 Data quality monitoring

The validation suite (`V1`–`V8`) is executed at the end of every cycle, and its quantitative outcomes are persisted in a dedicated table. A time series of the data quality measurements is plotted in the executive dashboard.

6.8.3 Model drift monitoring

Model drift is monitored by tracking, on a rolling basis, the proportions of device-months in each K-Means cluster and in each risk band. A statistically significant deviation triggers an alert and motivates a retraining.

6.8.4 Alerting

The alerting layer is implemented in `src/accent_fleet/monitoring/` and watches three signals: pipeline failures (status of the run log), data-quality regressions (validation suite) and model drift. When a threshold is crossed, a notification is sent to the operations channel (Slack webhook or e-mail).

6.8.5 Operations runbook

A short runbook documents the manual procedures: how to inspect the run log, how to trigger a backfill, how to restart the API or the dashboard, how to rotate the database credentials and how to renew the TLS certificate. The runbook is reproduced in Appendix C.

6.9 Production deployment workflow

6.9.1 Code deployment

The deployment of new code follows a simple workflow: a pull request is reviewed and merged on the `main` branch of the git repository; the operator pulls the new commits into `/opt/accent-fleet-analytics` on the VM; the dependencies are upgraded if needed; the API and the dashboard systemd units are restarted; and the next cron cycle exercises the new pipeline.

6.9.2 Database migrations

DDL changes follow a versioned migration pattern. Every change is shipped as a numbered SQL file under `sql/` and is applied by the bootstrap mode of the pipeline (or by a manual `psql` invocation under operator control). The migration scripts are idempotent: a re-execution does not corrupt the schema.

6.9.3 Model deployment

Model deployment relies on the artefact store described in Chapter 5. A new model is published as a versioned `joblib` artefact in a dedicated directory of the VM. The API picks up the most recent artefact at startup or upon receipt of an explicit reload signal.

6.9.4 Diagram of the deployment pipeline

The deployment pipeline is summarized in Figure 6.2.

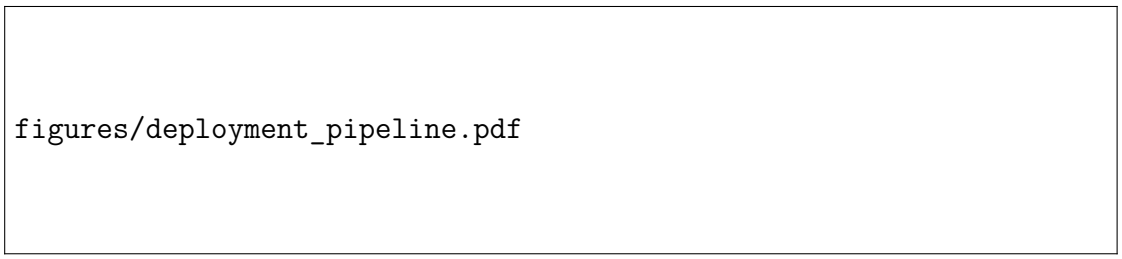

Figure 6.2: End-to-end deployment pipeline of the platform.

6.10 Conclusion

This chapter has formalized the deployment of the data engineering and machine learning platform on the Microsoft Azure infrastructure, with explicit attention to the analytical PostgreSQL warehouse fed from the converted MySQL dumps of Accent Tunisie, to the secure remote collaboration setup that allows the binomial of student

engineers to operate the platform from separate workstations (SSH tunnelling, IP allowlist on the Network Security Group, per-member service accounts), to the API and the BI dashboard, to the cron-driven automation, and to the layered monitoring and alerting plan. The end-to-end CRISP-DM cycle is therefore brought to its operational completion. The general conclusion that follows synthesizes the deliverables, the lessons learned and the perspectives opened by the project.

General Conclusion

The work documented in this thesis has covered the full CRISP-DM cycle of an industrial data engineering and machine learning project conducted at Accent Tunisie, on a multi-tenant telematics fleet of approximately six hundred GPS devices spread across four customer organizations. The starting point was an operational ecosystem composed of three MySQL databases — a raw hexadecimal telemetry stream populated by SIM-driven device frames, a decoded archive of `arch_xxxxx` tables retaining about six months of readable data, and a `user_xxxxx` master-data database carrying customers, vehicles and drivers — on top of which the company already exposed a set of descriptive dashboards but no advanced analytical layer. The endpoint is a production-grade analytical platform deployed on a Microsoft Azure virtual machine, refreshed every five minutes, equipped with an explicit MySQL-to-PostgreSQL conversion bridge, a cleaning rule engine, a dimensional warehouse, a registry of thirty-five behavioural features, a per-tenant K-Means segmentation, an Isolation Forest risk score, an API and a business intelligence dashboard.

The first half of the project has invested heavily in the data engineering layer. A dedicated bridge converts the `.sql` dump files received from Accent Tunisie — covering the slices of the `arch_xxxxx` archive and the `user_xxxxx` master data relevant to the analytical scope — into a PostgreSQL-compatible representation. Eleven data quality issues were identified during the data understanding phase and were formalized as cleaning rules executed at the warehouse boundary. A dimensional model with five dimensions, a bridge and eleven facts has stabilized the silver layer, on top of which the analytical marts and the unified ML view have been built. The watermark-driven incremental pipeline, hosted on the Azure VM and orchestrated by Prefect, has reduced the per-cycle processing time to a few seconds while preserving idempotency and replayability, and has done so without ever touching the operational MySQL workload of the company.

The second half of the project has built the analytical and serving layers. After a comparative study covering rule-based, statistical, classical machine-learning and deep-learning candidates, a pair K-Means + Isolation Forest, fitted per tenant, has been retained. The behavioural profiles obtained align with the operational vocabulary of the fleet managers; the rescaled risk score and its three operational bands feed the business intelligence dashboard and the API consumed by the consumer applications. A layered monitoring covers the pipeline health, the data quality and the model drift, with alerting hooked to the operations channel. Together, these deliverables move the

analytical centre of gravity of Accent Tunisie from a purely descriptive surface — *what happened?* — to a forward-looking decision-support layer — *which devices are likely to misbehave next month?*

A particular care has been given to the secure remote collaboration on the Azure infrastructure, dictated by the binomial nature of the project. Both members of the binomial worked from separate personal workstations and shared the analytical environment through their respective SSH key pairs. The PostgreSQL ports are not exposed to the public internet; access is mediated by SSH tunnels, themselves protected by a strict IP allowlist applied at the Azure Network Security Group level and limited to the binomial's workstations and the office network of Accent Tunisie. Per-member personal accounts coexist with a dedicated automation account on the VM, under controlled file permissions; the database service account is provisioned with the minimal privileges required by the pipeline.

Three lessons emerge from the project. *First*, the early formalization of the cleaning rules has been the single most effective decision: it has prevented the propagation of source-side anomalies into the modeling layer and has made every transformation auditable, an essential property when the source is an external dump file rather than a directly accessible database. *Second*, the per-tenant fitting protocol has been a decisive factor in obtaining operationally meaningful clusters; a global model would have been dominated by the most active tenant and would have failed to capture the heterogeneity of the regimes anticipated in the data understanding phase. *Third*, the strict separation between the operational MySQL stack of Accent Tunisie and the analytical PostgreSQL warehouse of the project has been more than an administrative convenience: it has freed the analytical work to iterate on cleaning rules, feature definitions and modeling choices without ever putting the customer-facing platform of the company at risk.

Three perspectives are open for future iterations. The first one is the integration of a labelled incident feed from the operations team of Accent Tunisie, which would unlock a supervised classification track and a quantitative calibration of the risk score against ground truth. The second one is the migration of the dump-based ingestion towards a streaming back-end (Kafka, Redpanda or a managed equivalent), in order to bring the latency below the minute and to support event-driven downstream consumers; this would naturally complement the existing streaming-style ingestion of the raw hexadecimal telemetry on the operational side. The third one is the introduction of a hierarchical or multi-task model capable of leveraging the structural similarities across tenants while preserving the tenant-specific regimes already identified.

Beyond the technical deliverables, the project has been a formative experience on the trade-offs between elegance and operational sustainability, and between methodological

rigour and the very concrete constraints of a binomial collaboration. Building a pipeline that runs successfully once is comparatively easy; building a pipeline that runs every five minutes for several months without manual intervention, on an infrastructure jointly operated by two student engineers from separate workstations, requires a different kind of engineering — one in which observability, idempotency, explicit failure modes and disciplined access control carry as much weight as algorithmic sophistication. The platform delivered in this thesis aims to incarnate that discipline.

Bibliography

[1] Peter J. Rousseeuw. “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis”. In: *Journal of Computational and Applied Mathematics* 20 (1987), pp. 53–65.

[2] David L. Davies and Donald W. Bouldin. “A cluster separation measure”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-1.2 (1979), pp. 224–227.

[3] Tadeusz Caliński and Jerzy Harabasz. “A dendrite method for cluster analysis”. In: *Communications in Statistics* 3.1 (1974), pp. 1–27.

[4] Sebastián Ramirez. *FastAPI Framework*. https://fastapi.tiangolo.com/. 2024.

Appendix A

SQL Exploration Scripts

This appendix references the SQL scripts used during the data understanding and the data preparation phases. The full versions are stored in the project repository under `scripts/sql/`.

A.1 Schema inspection

The script `01_schema_inspection.sql` returns the column-level metadata of every table in the staging and warehouse schemas, by querying `information_schema.columns`, `information_schema.table_constraints` and `information_schema.key_column_usage`. The result feeds the data dictionary and the schema map.

A.2 Data profiling

The script `02_data_profiling.sql` computes, for every column, descriptive statistics: row count, distinct count, null count, minimum, maximum, mean, standard deviation, and the 5^{th}, 50^{th}, 95^{th} percentiles. The script is parametrized by the table and the column name and is invoked through a shell wrapper that iterates over the catalogue.

A.3 Null analysis

The script `03_null_analysis.sql` produces the null rates of every column of the principal source tables, in a format directly consumable by the missing-value heatmap rendered by `fig_missing_heatmap.py`.

A.4 Distributions

The script `04_distributions.sql` computes per-column histograms with a configurable number of bins, using the `width_bucket` function of PostgreSQL. The result feeds the distribution plots produced in Chapter 3.

A.5 KPI extraction

The script `05_kpi_extraction.sql` computes the principal business KPIs at the tenant-month grain (total trips, total distance, average max speed, overspeed-per-100km, harsh-event-per-100km, idle ratio) directly from the marts. The result is consumed by the executive dashboard and by the figures of Chapter 5.

A.6 Metadata analysis

The script `06_metadata_analysis.sql` returns the size, the row count and the last analyse-timestamp of every table, by querying `pg_class`, `pg_stat_user_tables` and `pg_stat_all_indexes`. The script is used during the operational reviews of the platform.

A.7 Validation suite

The validation suite `99_validation_suite.sql` aggregates the eight checks `V1`–`V8` described in Chapter 4 and writes a synthetic report into a dedicated table consumed by the monitoring layer.

Appendix B

Python Scripts and Figure Generation

This appendix references the Python scripts used in the project. The full versions are stored under `scripts/python/`.

B.1 Automated extraction to CSV

The script `extract_to_csv.py` reads a manifest of SQL queries from `config/queries.yaml`, executes each query through a SQLAlchemy engine and writes the result as a timestamped CSV in `exports/`. The script accepts a `-dry-run` flag for unit-testing purposes.

B.2 Figure generation

The figures of the thesis are produced by self-contained Python scripts located in `scripts/python/figures/`. Each script reads its input either from the database (read-only credentials) or from a CSV produced by `extract_to_csv.py`, and writes a publication-quality PDF in `report/figures/`. The scripts and the figures they produce are listed in Table B.1.

Table B.1: Figure-generation scripts referenced in the thesis.

Script	Figure produced
`fig_crispdm_workflow.py`	Diagram of the CRISP-DM workflow.
`fig_architecture.py`	Logical architecture of the platform.
`fig_schema_map.py`	Schema map of the source tables.
`fig_missing_heatmap.py`	Missing-value heatmap.
`fig_distributions.py`	Distribution plots of the principal variables.
`fig_feature_importance.py`	Surrogate feature importance.
`fig_model_comparison.py`	Comparative dashboard of the candidate models.
`fig_confusion_matrix.py`	Confusion matrix between rule and Isolation Forest.
`fig_api_architecture.py`	Architecture of the API.
`fig_deployment_pipeline.py`	End-to-end deployment pipeline.
`fig_azure_topology.py`	Azure VM topology with NSG and SSH tunnel.

B.3 Modeling scripts

The modeling notebooks are stored under `notebooks/04_modeling/`. The notebook `01_device_behavior_clustering.ipynb` fits the per-tenant K-Means models and exports the artefacts; the notebook `02_anomaly_risk_score.ipynb` fits the per-tenant Isolation Forest models and computes the rescaled risk scores.

Appendix C

Azure Operations Runbook

This appendix collects the operational procedures of the Microsoft Azure environment hosting the platform. It is intended as a quick reference for the operations team of Accent Tunisie.

C.1 VM provisioning

C.1.1 VM specification

The platform runs on an Azure Linux virtual machine sized for a moderate analytical workload. The VM is provisioned in a Tunisian-or-European region of the Azure cloud, with a managed OS disk and an additional managed data disk dedicated to the PostgreSQL data directory. A static public IP address is associated with the VM.

C.1.2 Network Security Group

The Network Security Group (NSG) attached to the VM applies the following rules:

- **Inbound 22/tcp** (SSH): allowed only from the IP allowlist of the team and of the office network of Accent Tunisie.
- **Inbound 443/tcp** (HTTPS): allowed for the API and the dashboard, behind Nginx with a Let's Encrypt certificate.
- **Inbound 80/tcp** (HTTP): allowed only for the ACME challenge of Let's Encrypt; redirected to HTTPS otherwise.
- **Inbound 5432/tcp** (PostgreSQL): denied. PostgreSQL listens only on `localhost` and is reachable through SSH tunnels.

C.1.3 IP allowlist management

The IP allowlist is maintained as an explicit `ipset` attached to the NSG. Adding or removing a member of the team is a privileged operation that requires the approval of the platform owner. The history of the allowlist changes is preserved in the operations journal.

C.2 SSH access

C.2.1 Account types

Three account categories share the VM: personal accounts (one per team member, with SSH key authentication only and no shell password), an automation account (`azureuser`) used by cron and by the systemd services, and the database service account (`accent_pipeline`) used internally by the pipeline.

C.2.2 SSH tunnel

The SSH tunnel for PostgreSQL is opened by the team members on their workstation as shown in Listing C.1. Once the tunnel is open, a notebook or a database client connecting to `localhost:5432` reaches the analytical PostgreSQL on the VM.

```
ssh -i ~/.ssh/azure_id_ed25519 \
    -L 5432:localhost:5432 \
    -N \
    azureuser@<vm-public-ip>
```

Listing C.1: SSH tunnel for PostgreSQL access.

C.2.3 Key rotation

The SSH keys are rotated annually or upon departure of a team member, whichever comes first. The rotation procedure is recorded in the operations journal.

C.3 Pipeline operations

C.3.1 Cron entry

The cron entry under `/etc/cron.d/accent-fleet` triggers the incremental pipeline every five minutes (cf. Listing 6.2 of Chapter 6). The output of the run is piped to the system journal under the tag `accent-fleet`.

C.3.2 Inspecting the run log

The history of the pipeline runs is observed through the SQL query of Listing C.2.

```
SELECT run_id, mode, status, started_at, ended_at,
   rows_processed, error_message
FROM warehouse.etl_run_log
```

```
ORDER BY started_at DESC
LIMIT 20;
```

Listing C.2: Inspecting the run log.

C.3.3 Triggering a backfill

A backfill is triggered manually under operator control by Listing C.3. The backfill runs in a `tmux` session so that the operator can disconnect without interrupting the run.

```
tmux new -s backfill
cd /opt/accent-fleet-analytics
.venv/bin/python scripts/run_batch.py --mode backfill --chunk-days 30
# Detach with Ctrl-b, d
```

Listing C.3: Triggering a chunked backfill.

C.4 Database operations

C.4.1 Backup

A nightly logical backup is exported through `pg_dump` to `/opt/backups/`, rotated through a seven-day retention policy. A weekly full snapshot of the data disk is configured at the Azure infrastructure level and retained for four weeks.

C.4.2 Credential rotation

The credentials of the service account `accent_pipeline` are rotated quarterly. The rotation procedure updates the password in PostgreSQL, in the `.env` file of the VM, and restarts the API and the dashboard systemd units to reload the connection pool.

C.5 API and dashboard operations

C.5.1 Service management

The API and the dashboard are managed as systemd units `accent-api.service` and `accent-dashboard.service`. Listing C.4 shows the canonical commands to inspect, restart and follow the logs of these services.

```
sudo systemctl status  accent-api.service
sudo systemctl restart accent-api.service
```

```
sudo journalctl -u accent-api.service -f

sudo systemctl status  accent-dashboard.service
sudo systemctl restart accent-dashboard.service
sudo journalctl -u accent-dashboard.service -f
```

Listing C.4: Managing the API and dashboard systemd units.

C.5.2 TLS certificate renewal

The TLS certificate of Nginx is provisioned by Let's Encrypt and is renewed automatically by a daily timer. The renewal status is verified through `certbot certificates`.

C.6 Monitoring and alerting

C.6.1 Pipeline alerts

A failure of the pipeline (status `ERROR` in `etl_run_log`) triggers a notification on the operations channel. The notification carries the run identifier, the operating mode and the error message.

C.6.2 Data quality alerts

A failure of any critical check of the validation suite triggers a notification, with a link to the corresponding row of the validation table.

C.6.3 Drift alerts

A statistically significant deviation of the cluster proportions or of the risk band proportions, computed against a one-month baseline, triggers a notification accompanied by a link to the dashboard tile that visualizes the drift.